



AWS Whitepaper

AWS Best Practices for DDoS Resiliency



AWS Best Practices for DDoS Resiliency: AWS Whitepaper

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Abstract	i
Are you Well-Architected?	2
Introduction to denial of service attacks	3
Infrastructure layer attacks	5
UDP reflection attacks	5
SYN flood attacks	6
TCP middlebox reflection	8
Application layer attacks	8
Mitigation techniques	11
Best practices for DDoS mitigation	16
Infrastructure layer defense (BP1, BP3, BP6, BP7)	17
Amazon EC2 with Auto Scaling (BP7)	17
AWS Lambda and Serverless Compute (BP6)	18
Elastic Load Balancing (BP6)	19
Use AWS Edge locations for scale (BP1, BP3)	23
Application layer defense (BP1, BP2)	26
Attack surface reduction	41
Obfuscating AWS resources (BP1, BP4, BP5)	41
Security groups and network ACLs (BP5)	41
Protecting your origin (BP1, BP5)	42
Protecting API endpoints (BP4)	43
Operational techniques	45
Load testing	45
Metrics and alarms	45
AWS WAF traffic overview dashboard	52
Logging	53
Log correlation for DDoS resilient architectures	53
Visibility and protection management across multiple accounts	55
Incident response strategy and runbooks	57
Support	57
Conclusion	58
Contributors	59
Authors	59
Reviewers	59

Previous contributors	59
Further reading	60
Document revisions	61
Notices	63
AWS Glossary	64

AWS Best Practices for DDoS Resiliency

Publication date: **August 2026** ([Document revisions](#))

It's important to protect your business from the impact of Distributed Denial of Service (DDoS) attacks, as well as other cyberattacks. Keeping customer trust in your service by maintaining the availability and responsiveness of your application is high priority. You also want to avoid unnecessary direct costs when your infrastructure must scale in response to an attack. Amazon Web Services (AWS) is committed to providing you with the tools, best practices, and services to defend against bad actors on the internet. Using the right services from AWS helps ensure high availability, security, and resiliency.

In this whitepaper, AWS provides you with prescriptive DDoS guidance to improve the resiliency of applications running on AWS. This includes a DDoS-resilient reference architecture that can be used as a guide to help protect application availability. This whitepaper also describes different attack types, such as infrastructure layer attacks and application layer attacks. AWS explains which best practices are most effective to manage each attack type. In addition, the services and features that fit into a DDoS mitigation strategy are outlined, along with how each one can be used to help protect your applications.

This paper is intended for IT decision makers and security engineers who are familiar with the basic concepts of networking, security, and AWS. Each section has links to AWS documentation that provides more detail on the best practice or capability.

AWS detects over a million DDoS attacks per year and mitigates thousands on a daily basis against our customers. AWS recommends implementing these best practices proactively, as they significantly reduce the likelihood of business impact from DDoS events.

Are you Well-Architected?

The [AWS Well-Architected Framework](#) helps you understand the pros and cons of the decisions you make when building systems in the cloud. The six pillars of the Framework allow you to learn architectural best practices for designing and operating reliable, secure, efficient, cost-effective, and sustainable systems. Using the [AWS Well-Architected Tool](#), available at no charge in the [AWS Management Console](#) (sign-in required), you can review your workloads against these best practices by answering a set of questions for each pillar.

For more expert guidance and best practices for your cloud architecture—reference architecture deployments, diagrams, and whitepapers, see the [AWS Architecture Center](#).

Introduction to denial of service attacks

A Denial of Service (DoS) attack, or event, is a deliberate attempt to make a website or application unavailable to users, such as by flooding it with network traffic. Attackers use a variety of techniques that consume large amounts of network bandwidth or tie up other system resources, disrupting access for legitimate users. In its simplest form, a lone attacker uses a single source to carry out a DoS attack against a target, as shown in the following figure.



Figure 1 – A diagram depicting a DoS attack

In a DDoS attack, an attacker uses multiple sources to orchestrate an attack against a target. These sources can include distributed groups of malware infected computers, routers, IoT devices, and other endpoints. The following figure shows a network of compromised hosts that participate in the attack, generating a flood of packets or requests to overwhelm the target.

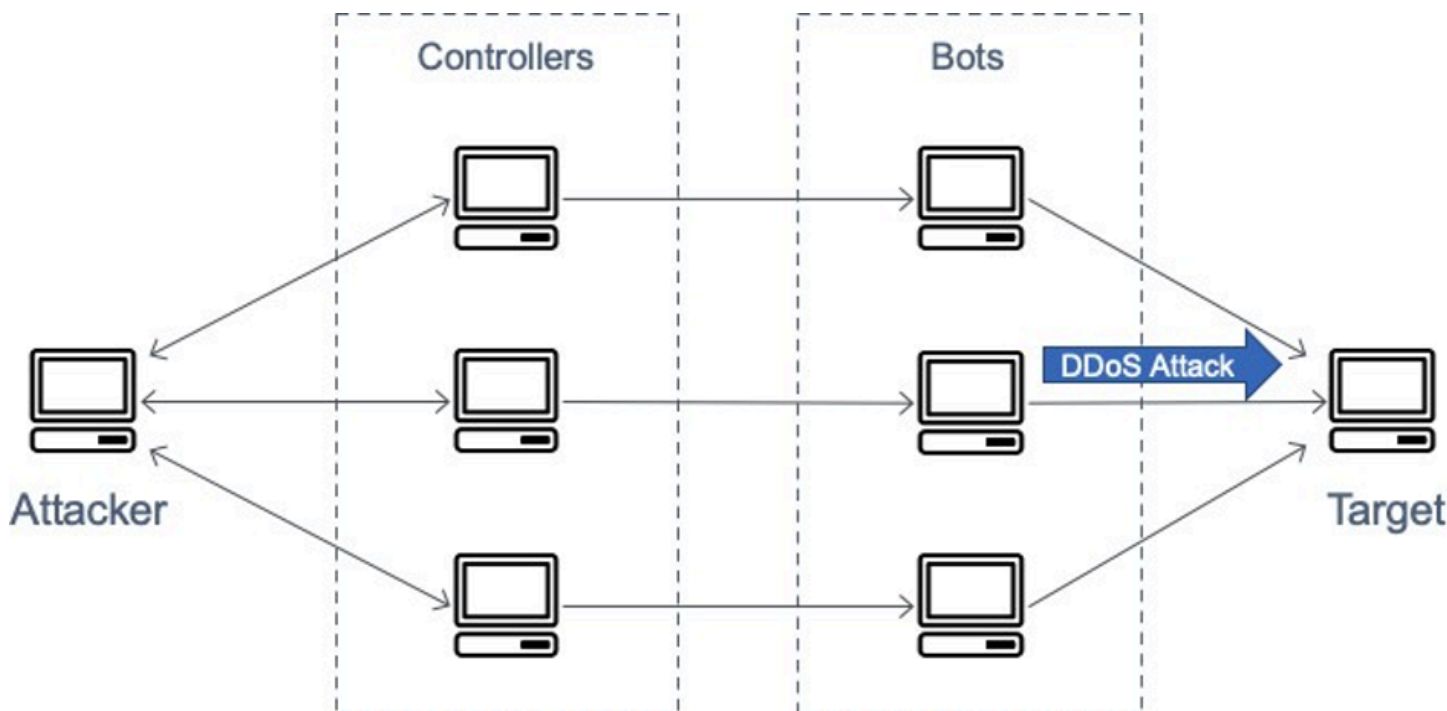


Figure 2 – A diagram depicting a DDoS attack

There are seven layers in the Open Systems Interconnection (OSI) model, described in the following table. DDoS attacks are most common at layers 3, 4, and 7.

- Layer 3 and 4 attacks correspond to the Network and Transport layers of the OSI model. Within this whitepaper, AWS refers to these collectively as *infrastructure layer attacks*.
- Layer 7 attacks correspond to the Application layer of the OSI model.

This paper discusses these attack types in the sections that follow.

Table 1 – OSI model

#	Layer	Unit	Description	Vector examples
7	Application	Data	Network process to application	HTTP floods, DNS query floods
6	Presentation	Data	Data representation and encryption	Transport Layer Security (TLS) abuse
5	Session	Data	Interhost communication	N/A
4	Transport	Segments	End-to-end connections and reliability	Synchronize (SYN) floods
3	Network	Packets	Path determination and logical addressing	User Datagram Protocol (UDP) reflection attacks
2	Data Link	Frames	Physical addressing	N/A

#	Layer	Unit	Description	Vector examples
1	Physical	Bits	Media, signal, and binary transmission	N/A

Infrastructure layer attacks

The most common DDoS attacks, User Datagram Protocol (UDP) reflection attacks and SYN floods, are *infrastructure layer attacks*. An attacker can use either of these methods to generate large volumes of traffic that can inundate the capacity of a network or tie up resources on systems such as servers, firewalls, intrusion prevention system (IPS), or load balancer. While these attacks can be easy to identify, to mitigate them effectively, you must have a network or systems that scale up capacity more rapidly than the inbound traffic flood. This extra capacity is necessary to either filter out or absorb the attack traffic freeing up the system and application to respond to legitimate customer traffic.

UDP reflection attacks

UDP reflection attacks exploit the fact that UDP is a stateless protocol. Attackers can craft a valid UDP request packet listing the attack target's IP address as the UDP source IP address. The attacker has now falsified (spoofed) the UDP request packet's source IP. The UDP packet contains the spoofed source IP and is sent by the attacker to an intermediate server. The server is tricked into sending its UDP response packets to the targeted victim IP rather than back to the attacker's IP address. The intermediate server is used because it generates a response that is several times larger than the request packet, effectively amplifying the amount of attack traffic sent to the target IP address.

The amplification factor is the ratio of response size to request size, and it varies depending on which protocol the attacker uses: DNS, Network Time Protocol (NTP), Simple Service Directory Protocol (SSDP), Connectionless Lightweight Directory Access Protocol (CLDAP), [Memcached](#), Character Generator Protocol (CharGen), or Quote of the Day (QOTD).

For example, the amplification factor for DNS can be 28 to 54 times the original number of bytes. So, if an attacker sends a request payload of 64 bytes to a DNS server, they can generate over 3400 bytes of unwanted traffic to an attack target. UDP reflection attacks are accountable for larger

volume of traffic in comparison to other attacks. The following figure illustrates the reflection tactic and amplification effect.

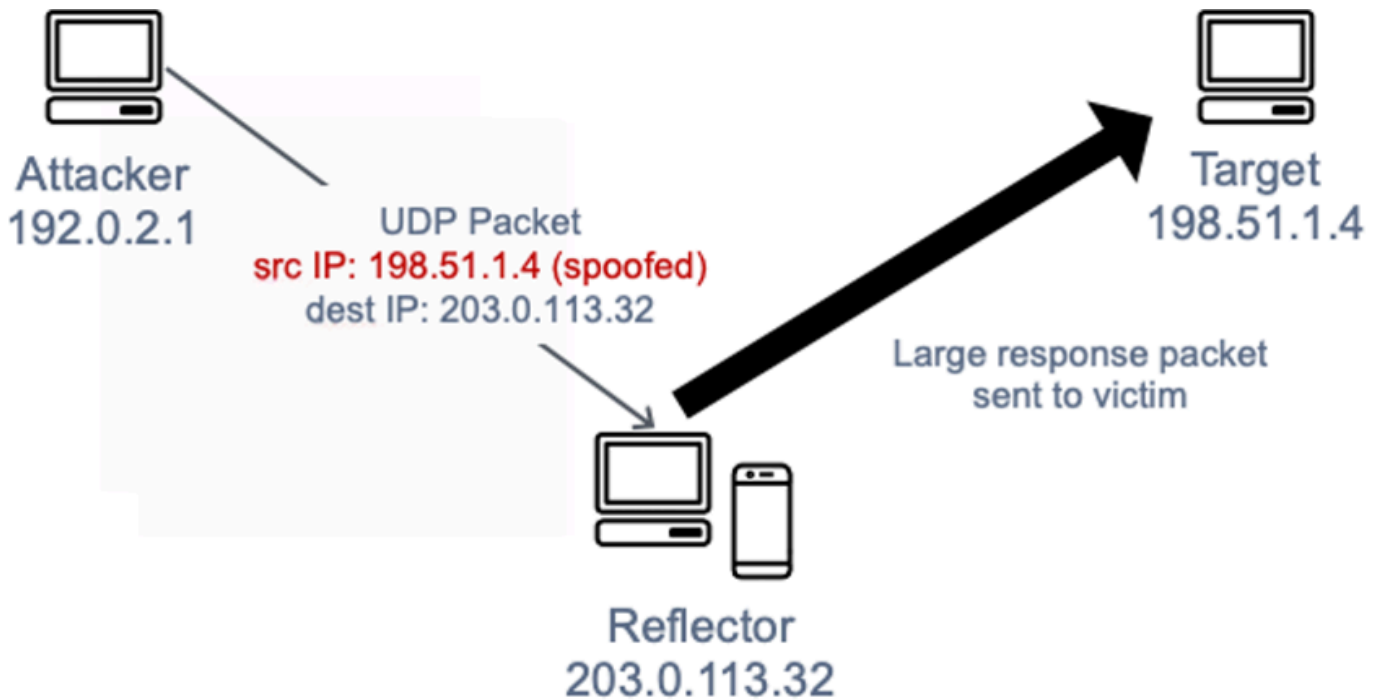


Figure 3 – A diagram depicting a UDP reflection attack

It should be noted that reflection attacks, while they provide attackers with amplification, require IP spoofing capability and as increasing numbers of network providers adopt Source Address Validation Everywhere (SAVE) or [BCP38](#), this capability is removed, requiring DDoS attackers to cease reflection attacks or to relocate to data centers and network providers who don't implement source address validation.

SYN flood attacks

When a user connects to a Transmission Control Protocol (TCP) service, such as a web server, their client sends a SYN packet. The server returns a synchronization-acknowledgement (SYN-ACK) packet, and finally the client responds with an acknowledgement (ACK) packet, which completes the expected three-way handshake. The following figure illustrates this typical handshake.

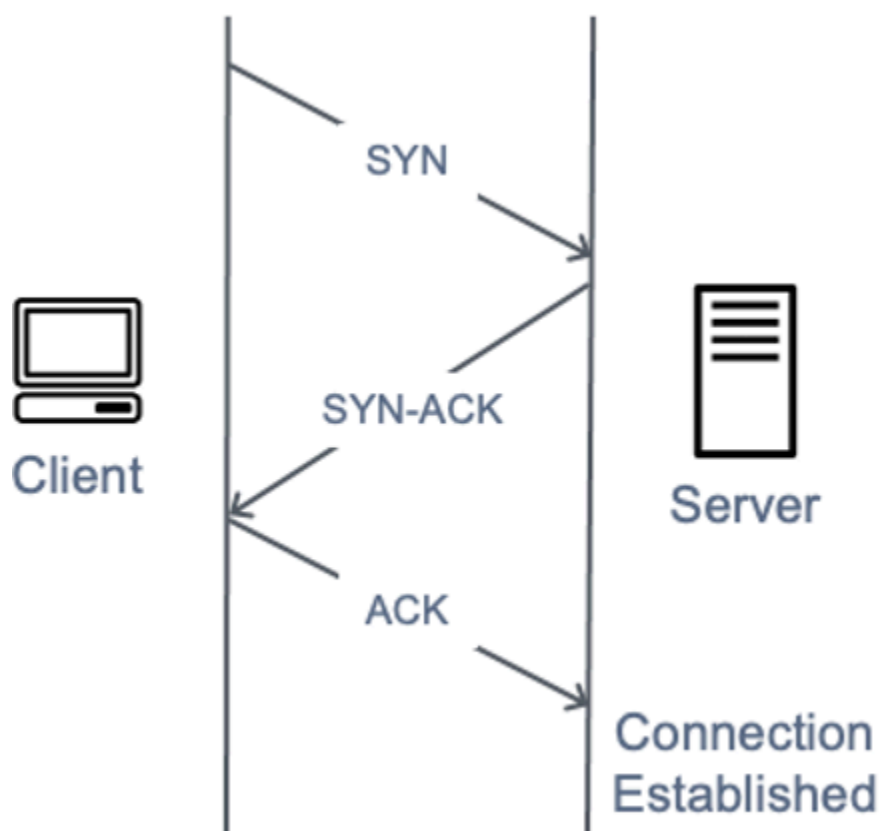


Figure 4 – A diagram depicting a SYN three-way handshake in a SYN flood attack

In a SYN flood attack, a malicious client sends a large number of SYN packets but never completes the TCP handshake. Modern operating systems implement SYN cookies by default to counter state table exhaustion.

You can improve the DDoS resilience of resources in your virtual private cloud (VPC) by ensuring that the security groups assigned to the resource—such as application load balancers (ALBs), classic load balancers (CLBs), and Elastic IPs (EIPs)—aren't configured to require Amazon Elastic Compute Cloud (Amazon EC2) [connection tracking](#). If a security group rule permits TCP or UDP flows for all traffic (0.0.0.0/0 or ::/0) to the listening port (80, 443, and so on), and there is a corresponding rule in the other direction that permits all response traffic (0.0.0.0/0 or ::/0) for all ports (0-65535) on both TCP and UDP, then that flow of traffic isn't tracked. This helps to avoid negative effects of this feature on an instance's packet throughput and enables an ALB to detect and scale based on the increase in traffic during a DDoS event.

TCP middlebox reflection

This relatively new attack vector was first disclosed in an [academic whitepaper](#) in August 2021 that explained how TCP non-compliance in network firewalls could result in these being tricked into becoming a TCP amplification vector. We've seen these attacks since early 2022 and continue to see them today. The amplification factor varies because of the different ways vendors have implemented this feature but can exceed Memcached UDP amplification.

Memcached amplification can reach an [estimated 51,000 time](#) the initial GET request by returning a large amount of UDP data, making it one of the highest known amplification factors—a 15-byte request can generate a response of up to 750 KB. This extreme amplification factor was exploited in February 2018 to launch attacks exceeding 1 Tbps. While the number of exposed memcached servers has decreased since then, it remains a significant amplification vector—and that TCP middlebox reflection can exceed this factor underscores the severity of this established and actively exploited attack vector.

Application layer attacks

An attacker might target the application itself by using a layer 7—application layer—attack. In these attacks—similar to SYN flood infrastructure attacks—the attacker attempts to overload specific functions of an application to make the application unavailable or unresponsive to legitimate users. Sometimes this can be achieved with very low request volumes that generate only a small volume of network traffic. This can make the attack difficult to detect and mitigate. Examples of application layer attacks include HTTP floods, cache-busting attacks, and WordPress XML-RPC floods.

- In an **HTTP flood attack**, an attacker sends HTTP requests that appear to be from a valid user of the web application. Some HTTP floods target a specific resource, while more complex HTTP floods attempt to emulate human interaction with the application. This can increase the difficulty of using common mitigation techniques such as request rate limiting.
- **Cache-busting attacks** are a type of HTTP flood that uses variations in the query string to circumvent content delivery network (CDN) caching. Instead of being able to return cached results, the CDN must contact the origin server for every page request, and these origin fetches cause additional strain on the application web server.
- With a **WordPress XML-RPC flood attack**, also known as a WordPress pingback flood, an attacker targets a website hosted on the WordPress content management software. The attacker misuses the [XML-RPC](#) API function to generate a flood of HTTP requests. The pingback feature

allows a website hosted on WordPress (Site A) to notify a different WordPress site (Site B) through a link that Site A has created to Site B. Site B then attempts to fetch Site A to verify the existence of the link. In a pingback flood, the attacker misuses this capability to cause Site B to attack Site A. This type of attack has a clear signature: "WordPress:" is typically present in the User-Agent of the HTTP request header.

There are other forms of malicious traffic that can impact an application's availability. *Scraper bots* automate attempts to access a web application to steal content or record competitive information, such as pricing. *Brute force* and *credential stuffing* attacks are programmed efforts to gain unauthorized access to secure areas of an application. These aren't strictly DDoS attacks, but their automated nature can look like a DDoS attack, and they can be mitigated by implementing some of the same best practices to be covered in this paper.

Application layer attacks can also target Domain Name System (DNS) services. The most common of these attacks is a *DNS query flood* in which an attacker uses many well-formed DNS queries to exhaust the resources of a DNS server. These attacks can also include a cache-busting component where the attacker randomizes the subdomain string to bypass the local DNS cache of any given resolver. As a result, the resolver can't take advantage of cached domain queries and must instead repeatedly contact the authoritative DNS server, which amplifies the attack.

If a web application is delivered over Transport Layer Security (TLS), an attacker can also choose to attack the TLS negotiation process. TLS is computationally expensive so an attacker, by generating extra workload on the server to process unreadable data (or unintelligible (ciphertext)) as a legitimate handshake, can reduce server's availability. In a variation of this attack, an attacker completes the TLS handshake but perpetually renegotiates the encryption method. An attacker can alternatively attempt to exhaust server resources by opening and closing many TLS sessions.

Application layer (L7) attacks have become increasingly prevalent in recent years, shifting from traditional network layer attacks. Modern Application layer attacks are characterized by:

- *Shorter duration but higher intensity:* Attacks have evolved to be more concentrated and shorter in duration, requiring faster detection and response capabilities.
- *Lower volume requirements:* Attackers can achieve significant impact with relatively low request volumes, making these attacks harder to detect through traditional traffic monitoring.

The evolution of these attacks has prompted the development of more sophisticated protection mechanisms, such as the [AntiDDoS AWS Managed Rules rule group \(Anti-DDoS AMR\)](#), which can detect and mitigate attacks within seconds while providing customizable protection levels.

Mitigation techniques

Some forms of DDoS mitigation are included automatically with AWS services. DDoS resilience can be improved further by using an AWS architecture with specific services—covered in the following sections—and by implementing additional best practices for each part of the network flow between users and your application.

You can use AWS services that operate from edge locations, such as [Amazon CloudFront](#), [AWS Global Accelerator](#), and [Amazon Route 53](#) to build comprehensive availability protection against all known infrastructure layer attacks. These services are part of the [AWS Global Edge Network](#), and can improve the DDoS resilience of your application when serving any type of application traffic from edge locations distributed around the world. You can run your application in any AWS Region and use these services to protect your application availability and optimize the performance of your application for legitimate end users.

Benefits of using Amazon CloudFront, Global Accelerator, and Amazon Route 53 include:

- Access to internet and DDoS mitigation capacity across the AWS Global Edge Network. This is useful in mitigating larger volumetric attacks, which can reach terabits per second (Tbps).
- [AWS Shield](#) DDoS mitigation systems are integrated with AWS edge services, reducing time-to-mitigate from minutes to sub second.
- Stateless SYN Flood mitigation verifies incoming connections using SYN cookies before passing them to the protected service. This is designed to help ensure that only valid connections reach your application while protecting your legitimate end users against false positives drops.
- Automatic traffic engineering systems that disperse or isolate the impact of large volumetric DDoS attacks. All these services isolate attacks at the source before they reach your origin, which means less impact on systems protected by these services.
- The AWS Threat research team curates and maintains a list of IP addresses by tracking malicious activities against customer's AWS resources. IP addresses exhibiting repeated malicious behavior are added to this list, which is automatically enforced through CloudFront edge protection systems to help safeguard customer resources.
- Application layer defense using CloudFront with [AWS WAF](#), without requiring changes to your existing origin architecture—whether hosted in an AWS Region or on-premises.
- [CloudFront](#) origin fetches are free from any AWS origin such as [Amazon Simple Storage Service](#) (Amazon S3), [Amazon EC2](#), or [Elastic Load Balancers](#), including origins in private subnets through

[VPC origins](#). Additionally, you aren't charged for HTTP requests or data transfer out for requests rejected by CloudFront or AWS WAF.

The following architecture diagrams include AWS Global Edge Network services. The BP labels in these diagrams see best practices that are detailed in later sections of this document.

The following figure illustrates a typical reference architecture web applications and maps its components to best practices that will be elaborated in the following sections of the chapter. These applications include the application layer of the Open Systems Interconnection (OSI) model and for that reason use CloudFront and AWS WAF at the edge for the protection.

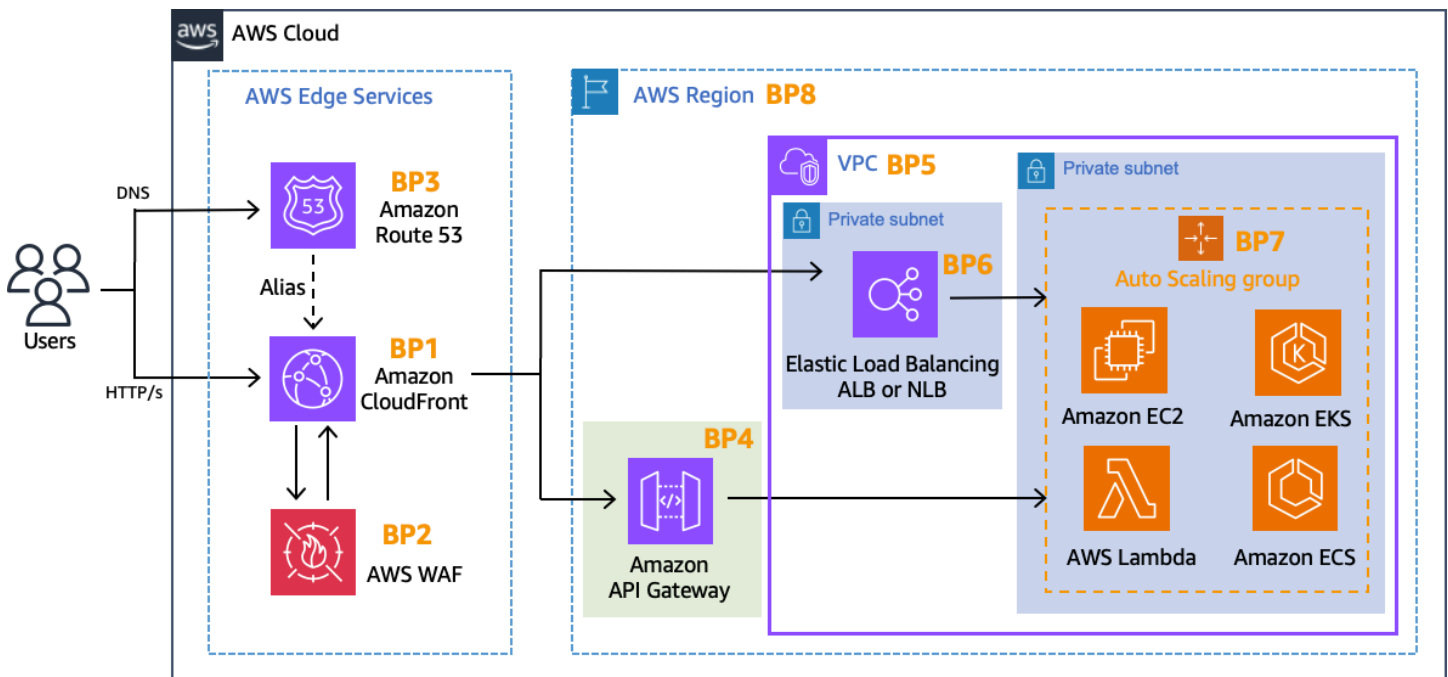


Figure 5 – DDoS-resilient reference architecture for web applications

The following figure illustrates a typical architecture for applications that operate at the transport layer of the OSI model and because of that the best practices include a different component for protection at the Edge.

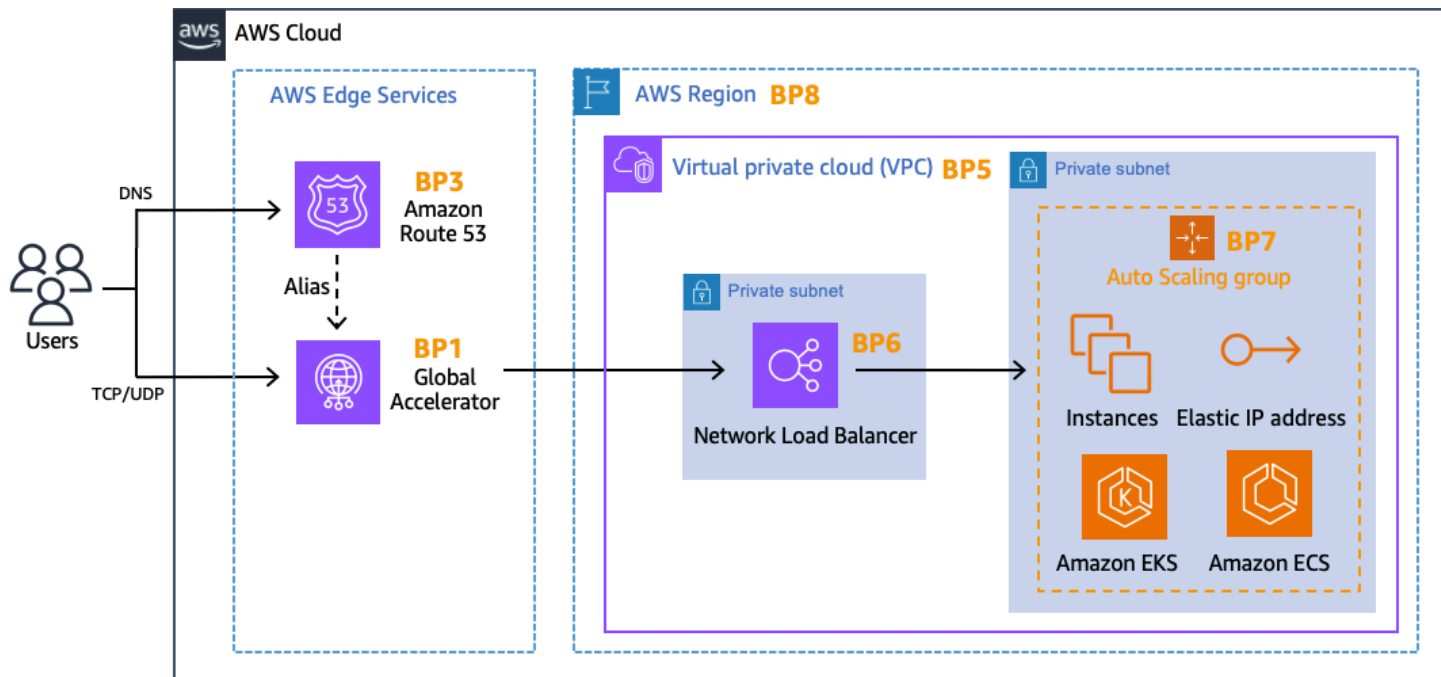


Figure 6 – DDoS-resilient reference architecture for TCP and UDP applications

With AWS service you can also protect applications that run inside corporate data centers, the following figure illustrates a collection of best practices applicable to that scenario.

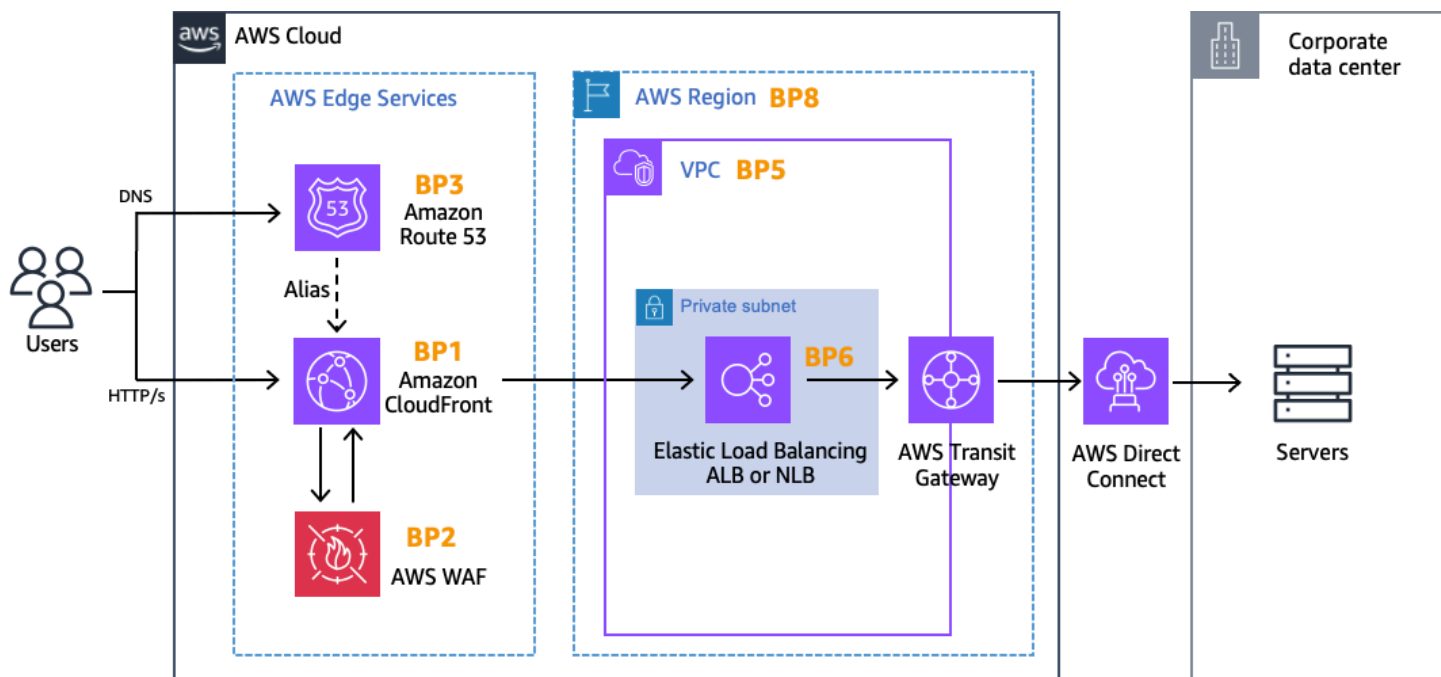


Figure 7 – DDoS-resilient reference architecture for web applications on-premises

These architectures include several AWS services that can help you improve your application's resiliency against DDoS attacks. The following table provides a summary of these services and the capabilities that they can provide. AWS has tagged each service with a best practice indicator (BP1, BP2) for easier reference within this document. For example, an upcoming section discusses the capabilities provided by Amazon CloudFront and Global Accelerator that includes the best practice indicator BP1.

Table 2 – Summary of best practices

	AWS Edge			AWS Region		
	Using Amazon CloudFront (BP1) with AWS WAF (BP2)	Using Global Accelerator (BP1)	Using Amazon Route 53 (BP3)	Using Application Load Balancer (BP6) with AWS WAF (BP2)	Using security groups and network ACLs in Amazon VPC (BP5)	Using Amazon Elastic Compute Cloud , Amazon EKS , and Amazon ECS with auto scaling, and Lambda (BP7)
Layer 3 (for example, UDP reflection) attack mitigation	✓	✓	✓	✓	✓	✓
Layer 4 (for example, SYN flood)	✓	✓	✓	✓		

	AWS Edge			AWS Region		
attack mitigation						
Layer 6 (for example, TLS) attack mitigation	✓	✓	✓	✓		
Reduce attack surface	✓	✓	✓	✓	✓	
Scale to absorb application layer traffic	✓	✓	✓	✓	✓	✓
Layer 7 (application layer) attack mitigation	✓		✓	✓		
Geographic isolation and dispersion of excess traffic and larger DDoS attacks	✓	✓	✓			

Another way to improve your readiness to respond to and mitigate DDoS attacks is by subscribing to [AWS Shield Advanced](#). Benefits of using AWS Shield Advanced include:

- Access to 24x7 specialized support from the [AWS Shield Response Team](#) (AWS SRT) for assistance with mitigating DDoS attacks that impact application availability, including an optional proactive engagement feature. It requires a Business Support+ or Enterprise Support subscription.
- Sensitive detection thresholds that route traffic into the DDoS mitigation system earlier and can improve time-to-mitigate attacks against Amazon EC2 (including elastic Load Balancer) or [Network Load Balancer](#), when used with an Elastic IP address
- [Automatic application layer DDoS mitigation](#) where Shield Advanced responds to detected DDoS attacks by creating, evaluating, and deploying custom AWS WAF rules
- Shield Advanced includes WAF request coverage at no additional cost (subject to usage thresholds). For current pricing details, see the [AWS Shield Pricing](#) page.
- Centralized management of security policies through [AWS Firewall Manager](#) at no additional cost.
- Cost protection that enables you to request a limited refund of scaling-related costs that result from a DDoS attack.
- [Protection groups](#) that enable you to bundle resources, providing a self-service way to customize the scope of detection and mitigation for your application by treating multiple resources as a single unit.
- DDoS attack visibility by using the [AWS Management Console](#), API, and [Amazon CloudWatch metrics](#) and [alarms](#).

This optional DDoS mitigation service helps protect applications hosted on any AWS Region. The service is available globally for CloudFront, Route 53, and Global Accelerator. Regionally, you can protect Application Load Balancer and Elastic IP addresses which allows you to protect [Network Load Balancer](#) (NLBs) or [Amazon EC2](#) instances.

For a complete list of AWS Shield Advanced features and for more information about [AWS Shield](#), see [How AWS Shield Advanced works](#).

Best practices for DDoS mitigation

In the following sections, each of the recommended best practices for DDoS mitigation is described in more depth. These practices are designed to help you build comprehensive DDoS protection

for both static and dynamic web applications using AWS services and architectural patterns. The sections are organized to guide you through implementing layers of defense, from edge protection to application-level security, to help ensure a robust DDoS-resilient architecture.

Infrastructure layer defense (BP1, BP3, BP6, BP7)

In a traditional datacenter environment, you can mitigate infrastructure layer DDoS attacks by using techniques like overprovisioning capacity, deploying DDoS mitigation systems, or scrubbing traffic with the help of DDoS mitigation services. On AWS, baseline DDoS mitigation capabilities are automatically provided by AWS Shield Standard at no additional cost; however, additional protections—such as AWS Shield Advanced, AWS WAF, and application-layer mitigations—require configuration and can incur additional charges. You can further optimize your application's DDoS resilience by making architecture choices that best use these capabilities and allow you to scale for excess traffic.

Key considerations to help mitigate volumetric DDoS attacks include ensuring that enough transit capacity and diversity are available and protecting AWS resources, like Amazon EC2 instances, against attack traffic.

To allow this high level of resilience, AWS recommends using [Amazon EC2 instances](#) with higher networking throughput that have an N suffix and support for [Enhanced Networking](#) with up to 200 Gbps of network bandwidth, for example, c7gn.16xlarge (Graviton3E, network-optimized) and c6in.32xlarge (Intel, network-optimized).

Amazon EC2 with Auto Scaling (BP7)

Another way to mitigate both infrastructure and application layer attacks is to operate at scale. If you have web applications, you can use load balancers to distribute traffic to Amazon EC2 instances that are overprovisioned or configured to automatically scale. These instances can handle sudden traffic surges that occur for any reason, including a flash crowd or an application layer DDoS attack. You can set [Amazon CloudWatch alarms](#) to initiate [Auto Scaling](#) to automatically scale the size of your Amazon EC2 fleet in response to events that you define, such as CPU, RAM, Network I/O, and even custom metrics. That approach is illustrated in the following diagram.

This approach protects application availability when there's an unexpected increase in request volume. When using Amazon CloudFront, [Application Load Balancer](#), or [Network Load Balancer](#) with your application, TLS negotiation is handled by the distribution (CloudFront) or by the load balancer. These features help protect your instances from being impacted by TLS-based attacks by scaling to handle legitimate requests and TLS abuse attacks.

For more information about using Amazon CloudWatch to invoke Auto Scaling, see [Monitoring Amazon CloudWatch metrics for your Auto Scaling groups and instances](#).

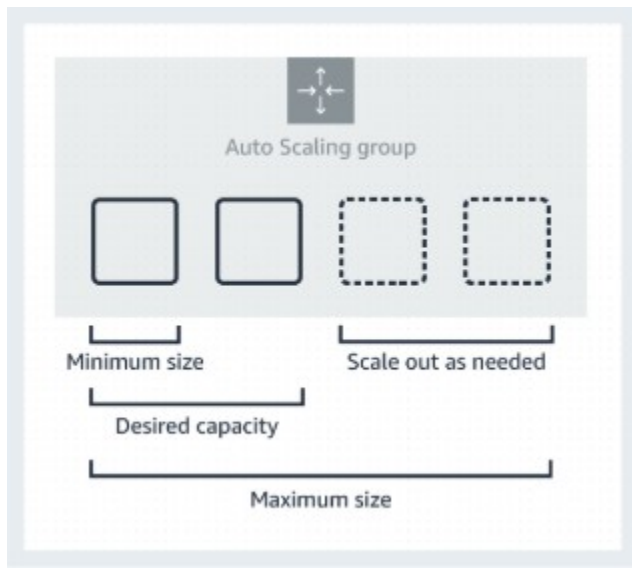


Figure 8 – EC2 Auto Scaling group configuration showing minimum, desired, and maximum instance capacity ranges

[Amazon EC2 Auto Scaling](#) groups are collections of Amazon EC2 instances that provide resizable compute capacity so that you can quickly scale up or down as requirements change. You can scale horizontally by automatically adding instances to your application by [scaling the size of your Amazon EC2 Auto Scaling group](#), and you can scale vertically by using larger EC2 instance types.

By using [Amazon RDS Proxy](#), you can allow your applications to pool and share database connections to improve their ability to scale and handle unpredictable surges in database traffic. You can also enable storage auto-scaling for an Amazon RDS database instance. See [Managing capacity automatically with Amazon RDS storage autoscaling](#) for more information.

AWS Lambda and Serverless Compute (BP6)

AWS Lambda automatically scales by running additional instances of your functions in response to incoming requests. Each function instance handles one request at a time, and Lambda can scale to thousands of concurrent executions within seconds.

For DDoS resilience considerations with Lambda:

- [Reserved concurrency](#) – Set a reserved concurrency limit to help ensure your function has capacity available, even if other functions in the account are consuming concurrency.

- **[Provisioned concurrency](#)** – Pre-initialize function instances to reduce cold start latency during traffic surges. This is particularly useful for latency-sensitive applications that might experience sudden traffic increases.
- *Function URL throttling* – If using [Lambda function URLs](#), configure reserved concurrency to act as a throttle against excessive requests.
- **[Amazon API Gateway](#) integration** – When Lambda is behind API Gateway, you benefit from the API Gateway built-in throttling (default 10,000 requests per second per Region) and can add AWS WAF for additional protection.

Be aware that Lambda has account-level concurrency limits. A DDoS attack targeting one function could consume concurrency needed by other functions. Use reserved concurrency to isolate critical functions.

Elastic Load Balancing (BP6)

Large DDoS attacks can overwhelm the capacity of a single Amazon EC2 instance. With [Elastic Load Balancing](#), you can reduce the risk of overloading your application by distributing traffic across many backend instances. Elastic Load Balancing can scale automatically, allowing you to manage larger volumes when you have unanticipated extra traffic, for example, because flash crowds or DDoS attacks. For applications built within an Amazon VPC, there are different types of ELBs to consider, for example: Application Load Balancer (ALB) and Network Load Balancer (NLB).

Best practices for web applications (HTTP/HTTPS)

For web applications, you can use the [Application Load Balancer](#) to route traffic based on content and accept only well-formed web requests, as shown in the following diagram. Application Load Balancer blocks many common DDoS attacks, such as SYN floods or UDP reflection attacks, protecting your application from the attack. Application Load Balancer automatically scales to absorb the additional traffic when these types of attacks are detected. Scaling activities due to infrastructure layer attacks are transparent for AWS customers and don't affect your bill.

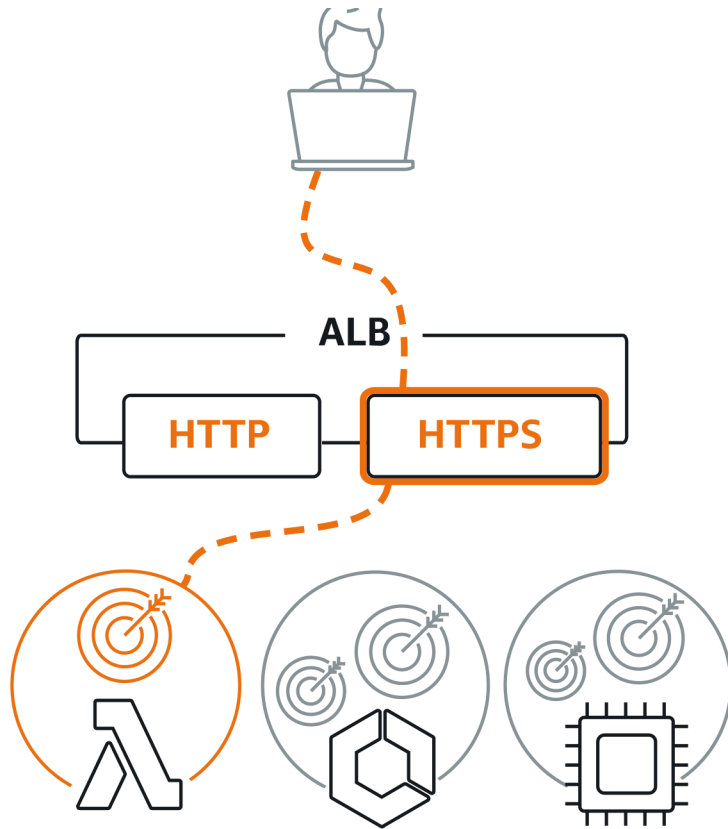


Figure 9 – Application Load Balancers provide a flexible feature set for HTTP and HTTPS applications

For more information about protecting web applications with Application Load Balancer, see [Getting Started with Application Load Balancers](#).

Best practices for TCP and UDP applications

For TCP, UDP, and TLS applications, you can use [Network Load Balancer](#) to route traffic to targets (for example, Amazon EC2 instances) at ultra-low latency, as shown in the following diagram. Choose a network load balancer (NLB) when you need ultra-high performance, TLS offloading at scale, centralized certificate deployment, support UDP, and static IP address for your applications. One key consideration with Network Load Balancer is that any TCP or UDP traffic that reaches the load balancer on a valid listener will be routed to your targets, not absorbed, however this doesn't apply for TLS-listeners that terminate the TCP connection.

Note

For NLB with TCP listeners, it's recommended to deploy these behind an AWS Global Accelerator to help protect against SYN floods. Global Accelerator has an inbuilt SYN proxy and connections made through the NLB are automatically tracked.

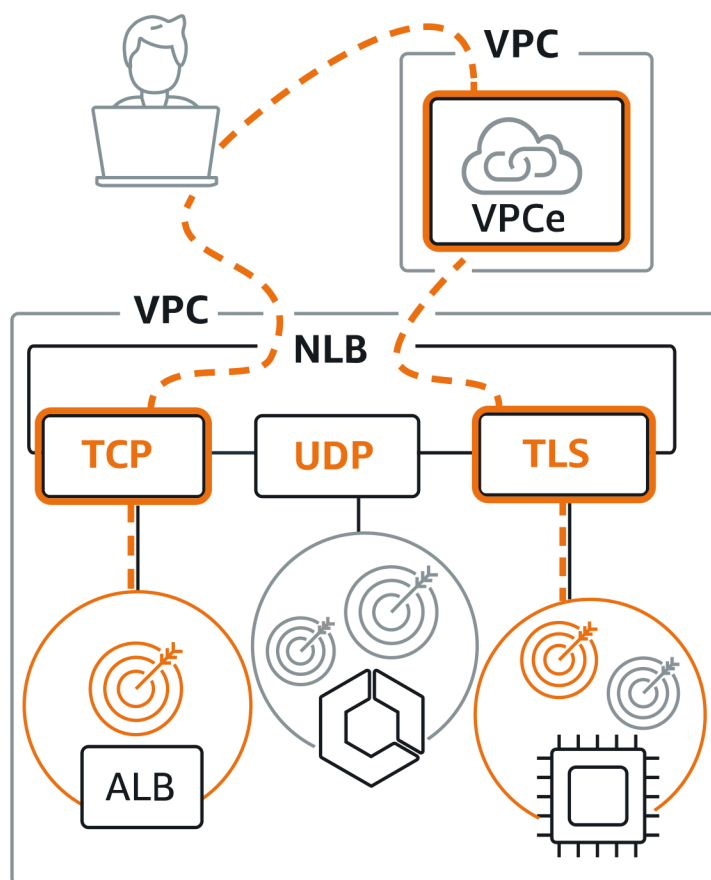


Figure 10 – Network load balancers provide high performance and a suite of support features

You can use Shield Advanced to configure DDoS protection for Elastic IP addresses. When an Elastic IP address is assigned per Availability Zone to the NLB, Shield Advanced will apply the relevant DDoS protections for the NLB traffic.

For more information about protecting TCP and UDP applications with Network Load Balancer, see [Create a Network Load Balancer](#).

Note

Depending on the security group configuration of your Application Load Balancer, it requires the resource using the security to group to use connection tracking to track information about traffic, this can affect the load balancer's ability to process new connections, because the number of tracked connections is limited. A security group configuration that contains an ingress rule accepting traffic from any IP address (for example, `0.0.0.0/0` or `::/0`) but doesn't have a corresponding rule to allow the response traffic, causes the security group to use connection tracking information to allow the response traffic to be sent. In an event of a DDoS attack, the maximum number of tracked connections can be exhausted. To improve the DDoS resilience of your public-facing Application Load Balancer, ensure that the security group associated with your load balancer is configured to not use connection tracking (untracked connections), so the flow of traffic isn't subject to connection tracking limits.

For this, configure your security group with a rule that allows inbound TCP traffic on your listener ports (for example, 80 and 443) from any IP address (`0.0.0.0/0` or `::/0`), and add a corresponding rule in the outbound direction allowing this resource to send the response traffic (allow outbound range for any IP address `0.0.0.0/0` or `::/0`) for all ports (0-65535), so the response traffic is allowed based on the security group rule, and not on tracking information. With this configuration, Classic and Application Load Balancer aren't subject to exhaust connection tracking limits that might affect establishing new connections to its load balancer nodes and allows it to scale based on the increase in traffic in the event of a DDoS attack. More information about untracked connections can be found at: [Security group connection tracking: Untracked connections](#).

Avoiding security group connection tracking protects you on both paths. For DDoS traffic from allowed sources, untracked connections keep the connection tracking table from being exhausted. For DDoS traffic from sources the security group doesn't allow, the traffic is dropped at the security group, so it never creates connection tracking entries in the first place.

For EC2 instances behind load balancers, configure security group rules to allow all traffic from the load balancer's security group on the target port. This creates untracked connections that don't consume connection tracking table entries, preventing table exhaustion during traffic surges or DDoS attacks.

Use AWS Edge locations for scale (BP1, BP3)

Access to highly scaled, diverse internet connections can significantly increase your ability to optimize latency and throughput to users, to absorb DDoS attacks, and to isolate faults while minimizing the impact on your application's availability. AWS edge locations provide an additional layer of network infrastructure that provides these benefits to any application that uses Amazon CloudFront, Global Accelerator and Amazon Route 53. With these services, you can comprehensively protect on the edge your applications running from AWS Regions.

Web application delivery at the edge (BP1)

You can use Amazon CloudFront to deliver your entire website including static, dynamic, streaming, and interactive content. Persistent connections and variable time-to-live (TTL) settings can be used to offload traffic from your origin, even if you aren't serving cacheable content. Use of these CloudFront features reduces the number of requests and TCP connections back to your origin, helping protect your web application from HTTP floods.

CloudFront only accepts well-formed connections, which helps prevent many common DDoS attacks, such as SYN floods and UDP reflection attacks, from reaching your origin. DDoS attacks are also geographically isolated close to the source, which prevents the traffic from impacting other locations. These capabilities can greatly improve your ability to continue serving traffic to users during large DDoS attacks. You can use CloudFront to protect an origin on AWS or elsewhere on the internet.

If you're using [Amazon Simple Storage Service](#) (Amazon S3) to serve static content on the internet, AWS recommends you use Amazon CloudFront to protect your bucket providing the following benefits:

- Restricts access to the Amazon S3 bucket so that it's not publicly accessible.
- Helps ensure sure that viewers (users) can access the content in the bucket only through the specified CloudFront distribution—that is, prevents them from accessing the content directly from the bucket.
- Lower latency for end users by serving content from edge locations closer to them.
- Reduced cost for requests to S3 origins (data transfer from S3 to CloudFront is free).

To achieve this, configure CloudFront to send authenticated requests to Amazon S3, and configure Amazon S3 to only allow access to authenticated requests from CloudFront. CloudFront provides

two ways to send authenticated requests to an Amazon S3 origin: origin access control (OAC) and origin access identity (OAI). We recommend using OAC because it supports:

- All Amazon S3 buckets in all AWS Regions, including opt-in Regions launched after December 2022
- Amazon S3 [server-side encryption](#) with AWS KMS (SSE-KMS)
- Dynamic requests (PUT and DELETE) to Amazon S3

For more information about OAC and OAI, see [Restricting access to Amazon S3 origin](#).

For more information about protecting and optimizing the performance of web applications with Amazon CloudFront, see [Getting Started with Amazon CloudFront](#) and the diagram in Mitigation Techniques.

Protect network traffic further from your origin using AWS Global Accelerator (BP1)

You can implement a DDoS resilient architecture that provides many of the same benefits as the web application delivery at the edge best practices, even if your application uses protocols not supported by CloudFront or you're operating a web application that requires global static IP addresses.

Global Accelerator is a networking service that improves availability and performance of users' TCP and UDP traffic by up to 60%. This is accomplished by ingressing traffic at the edge location closest to your users and routing it over the AWS global network infrastructure to your application, whether it runs in a single or multiple AWS Regions.

Global Accelerator routes TCP and UDP traffic to the optimal AWS endpoint based on the user's geographic location, endpoint health, and configured endpoint weights, reducing latency and improving availability. If there is an application failure, Global Accelerator provides failover to the next best endpoint. Global Accelerator uses the vast capacity of the AWS global network and integrations with Shield—such as a stateless SYN proxy that challenges new connection attempts with SYN cookies, dropping spoofed traffic that can't complete the TCP handshake—to protect applications.

For example, you might require IP addresses that your end users can add to the allow list in their firewalls and aren't used by any other AWS customers. In these scenarios you can use Global Accelerator to protect web applications running on Application Load Balancer and in conjunction with AWS WAF to also detect and mitigate web application layer request floods.

For more information about protecting and optimizing the performance of network traffic using Global Accelerator, see [Getting started with Global Accelerator](#) and the diagram *DDoS-resilient reference architecture for TCP and UDP applications* in [Mitigation techniques](#).

Domain name resolution at the edge (BP3)

Using Route 53 for DNS availability

Amazon Route 53 is a highly available and scalable DNS service that you can use to direct traffic to your web application. It includes advanced features like [Traffic Flow](#), [Health Checks and Monitoring](#), [Latency-Based Routing](#), and [Geo DNS](#). You can use these advanced features to control how the service responds to DNS requests to improve the performance of your web application and to avoid site outages. It's the only AWS service that has a 100% data plane availability SLA.

Amazon Route 53 uses techniques such as [shuffle sharding](#) and [anycast striping](#), that can help users access your application even if the DNS service is targeted by a DDoS attack.

With shuffle sharding, each name server in your delegation set corresponds to a unique set of edge locations and internet paths. This provides greater fault tolerance and minimizes overlap between customers. If one name server in the delegation set is unavailable, users can retry and receive a response from another name server at a different edge location.

Anycast striping allows each DNS request to be served by the most optimal location, dispersing the network load and reducing DNS latency. This provides a faster response for users. Additionally, Amazon Route 53 can detect anomalies in the source and volume of DNS queries and prioritize requests from users that are known to be reliable.

For more information about using Amazon Route 53 to route users to your application, see [Getting Started with Amazon Route 53](#).

Configuring Route 53 for cost protection from NXDOMAIN attacks

NXDOMAIN attacks occur when attackers send a flood of requests to a hosted zone for non-existent sub-domains, often using known legitimate resolvers. The purpose of these attacks might be to impact the cache of the recursive resolver or the availability of the authoritative resolver, or could be a form of DNS reconnaissance to try to discover hosted zone records. Using Route 53 for your authoritative resolver mitigates the risk of availability/performance impact, however the result can be an increase in the per-query cost of Route 53. To protect against cost increases, take advantage of [Route 53 pricing](#) in which DNS queries are free when both of the following are true:

- The domain (example.com) or subdomain name (store.example.com) and the record type (A) in the query match an alias record.
- The alias target is an AWS resource other than another Route 53 record.

Create a [wildcard record](#), for example, *.example.com with a type A (Alias) pointing at an AWS resource such as an Amazon S3 bucket or CloudFront distribution, so that when a query for qwerty12345.example.com is made, the IP of the resource will be returned and you will not be charged for the query.

Application layer defense (BP1, BP2)

Many of the techniques discussed so far in this paper are effective at mitigating the impact that infrastructure layer DDoS attacks have on your application's availability. To also defend against application layer attacks, you need to implement an architecture that allows you to specifically detect, scale to absorb, and block malicious requests. This is an important consideration because network-based DDoS mitigation systems are generally ineffective at mitigating complex application layer attacks.

Detect and filter malicious web requests (BP1, BP2)

When your application runs on AWS, you can use Amazon CloudFront (and its HTTP caching capability), AWS WAF, and Shield Advanced Automatic Application layer protection to help prevent unnecessary requests reach your origin during application layer DDoS attacks.

This section outlines a defense-in-depth strategy to protect your origin infrastructure using two complementary approaches. First, AWS WAF acts as the primary defense layer by filtering and blocking malicious requests before they reach your architecture. Second, HTTP caching serves as an additional protective layer by reducing the load on your infrastructure behind CloudFront. By implementing both AWS WAF filtering and strategic HTTP caching, you create multiple layers of defense that significantly reduce the risk of malicious traffic impacting your origin servers while optimizing legitimate request handling.

Amazon CloudFront

Amazon CloudFront is a content delivery network (CDN) that helps protect HTTP/S-based applications in the following ways:

- CloudFront protects your backend services by validating connections at multiple network layers. Each request must establish a proper TCP or UDP (for QUIC and HTTP/3) session, complete a

TLS handshake (for HTTPS), and provide a well-formed HTTP gRPC request. This multi-layer validation filters out malformed or incomplete connection attempts before they reach your origin.

- CloudFront edge locations provide automatic, always-on mitigation of network and transport layer (Layer 3 and 4) attacks, protecting both the CDN service and your origin infrastructure.
- CloudFront automatically closes connections from slow reading or slow writing attackers (for example, [Slowloris](#)), or novel attacks such as HTTP/2 rapid reset.
- CloudFront provides built-in protection against common HTTP protocol attacks, including HTTP request smuggling, HTTP desync attacks, and request splitting.
- CloudFront WAF capacity is scaled to match the CloudFront points of presence (POP) capacity—the capacity constraints of Regional WAF don't apply.
- The CloudFront global edge network provides massive absorption capacity for traffic spikes, regardless of where your origin is hosted. Whether your backend infrastructure is in AWS, another cloud provider, or on-premises, CloudFront can be combined with properly configured WAF rules and effective caching strategies to absorb large-scale request floods that would otherwise overwhelm backend origin services.

HTTP caching

CloudFront

Amazon CloudFront caching capabilities are a powerful tool for protecting your origin servers' capacity, even with dynamic content delivery. By implementing strategic caching policies, even with short time to live (TTL), you can significantly reduce the load on your origin infrastructure.

When content is served from the CloudFront [cache hierarchy](#)—edge POPs (points of presence closest to users), Regional Edge Caches (RECs, larger consolidated caching layers that sit between POPs and your origin), and Origin Shield (an additional centralized caching layer that reduces origin load by acting as a single point of contact for cache fills)—requests might be served as cache objects from downstream peers in the network, preventing volume from reaching your origin.

For content that isn't yet cached or has expired, CloudFront [request collapsing](#) further protects origins by consolidating multiple simultaneous requests into a single origin request. This combination of edge caching and request collapsing means that even during high-traffic events or volumetric request floods, only a fraction of requests need to reach your origin service, making CDN caching an essential strategy for origin capacity preservation.

There's a common misconception among content owners that their content isn't CDN-cacheable, when in fact it can be cached or be made to be cacheable. Even when dealing with real dynamic content delivered through asynchronous requests, the main page structure itself can and should be cached. Maximizing the [cache hit ratio](#) is crucial for performance and DDoS resilience—it can be the determining factor between whether an HTTP request flood attack impacts your service or not.

Important

Page cacheability should be established as a non-functional requirement during the initial planning stages of any website deployment.

Did you know? Statistically, the '/' URI is the most commonly targeted URI by bad actors in HTTP request floods.

The following are some solutions to consider for some common perceived blockers to caching static content:

- For web pages that require immediate updates when new content is published, content freshness can be managed through multiple strategies, using HTTP standard [Cache-Control response header](#) directives, CloudFront [caching policy](#) settings, or a combination of the two as follows:
 - Implement a split TTL approach: Set 0 TTL for viewers using the Cache-Control response header `max-age=0` directive. This means that clients will always check for content freshness on page re-load. Set longer TTLs at the CDN level using a non-zero [minimum TTL](#) in the relevant distribution caching policy (using the managed [CachingOptimized](#) cache policy makes sense here). You can also optionally add the [s-maxage Cache-Control](#) response header directive at the origin to increase the CDN caching time above the minimum TTL.
 - Use microcaching with minimum TTL values as low as 1 or 2 seconds for non-personalized but highly dynamic content such as election results or live streaming manifests.
 - Set Etag or Last-Modified response headers to enable [conditional requests](#). When content hasn't changed, CloudFront responds with an [HTTP 304 \(Not Modified\)](#) status and a zero-byte response body, which optimizes page load times and minimizes Data Transfer Out charges.
 - Use CloudFront [cache invalidations](#), which can be automatically triggered through content management system integrations during publish events—be aware that there are limits in terms of requests per second for these cache invalidation APIs.
- For static assets such as images, scripts, and other page resources, implement unique file versioning with higher TTL values (1 year or more). Instead of updating files in place, publish new

versions with unique URIs or query strings, and update their references in HTML. This strategy provides optimal delivery efficiency and maximum offload across browser caches, CDN cache, and origin services while ensuring immediate content updates through reference changes rather than cache invalidations.

- For sites with mixed content, create multiple cache behaviors with appropriate cache policies to maximize DDoS resilience. While static content should be aggressively cached, truly dynamic endpoints that can't be cached should be identified early and protected with additional controls such as scoped-down AWS WAF [rate-based rules](#) with lower limits than any rate-based rules that are broadly applied.
- Non-personalized HTML with Cache-Control header directives like no-cache, no-store can be cached on CDN by using a non-zero [Minimum TTL](#) in the relevant CloudFront cache behavior.
- In a situation where the origin performance has been adversely impacted, consider adding the [stale-while-revalidate](#) Cache-Control directive, so that users no longer need to wait for responses from the origin. It's ideal for content that refreshes frequently or unpredictably, or where content requires significant time to regenerate, and where having the latest version of the content isn't essential.
- If you have an adaptive website and serve different pages based on device-type, you can configure caching based on device type by updating both the cache policy and [origin request policy](#) to use [device type headers](#).
- The [stale-if-error directive](#) enhances the user experience and improves availability by serving stale content when origins return an error.
- Most major content management systems (CMS) have a plug-in to replicate static content to Amazon S3.

Note

If you [forward all headers to the origin](#), caching is disabled regardless of other cache settings.

Request collapsing

Request collapsing occurs when multiple users simultaneously request the same content that isn't in cache. CloudFront intelligently manages these requests by sending only one to the origin server, while briefly pausing all the others. After CloudFront receives the origin's response, it serves this

cached content to all waiting requests, preventing a flood of duplicate requests from the same caching instance from overwhelming your origin server during a typical DDoS attack.

The following configurations prevent request collapsing from occurring:

- The minimum TTL is 0
- Cookie forwarding is enabled in [the cache policy](#), [the origin request policy](#), or the legacy cache settings

Note

A request flood attack in conjunction with non-collapsed but otherwise cacheable requests, can be sufficient to overwhelm some origins, especially those that are unable to dynamically scale.

Origin shield

Enabling features like [CloudFront Origin Shield](#) can help further reduce the load on your origin by enabling a centralized caching layer. This can be useful for low-capacity origins that are unable to dynamically scale.

Error pages and error caching

CloudFront [caches some 4xx and 5xx status codes](#) returned by your origin for a default duration of 10 seconds. While it might seem illogical to cache server errors, doing so can prevent a snowball effect of viewer retries from an error response further degrading an origin's health. Another option is to use the `stale-if-error` Cache-Control directive for content that is cached.

In addition, you can now use CloudFront to create [custom error pages](#). If you choose to configure these, we recommend:

- Storing error pages in an alternate origin, such as an Amazon S3 bucket, so that CloudFront can access them, even when your other origins are returning errors.
- Enabling CDN caching for the error pages with a longer minimum TTL than the error caching TTL to minimize requests to the origin containing the error pages.

Amazon EC2-based caching

There are some use cases, such as requiring regional TLS-termination, where customers don't want to use a global CDN like CloudFront. In this case, it's worth setting up a dynamically scaled EC2 instance running a cache such as Varnish, to protect the content source.

AWS WAF

By using AWS WAF, you can configure web access control lists (web ACLs) on your global CloudFront distributions or regional resources to filter, monitor and block requests based on request signatures. To determine whether to allow or block requests, you can consider factors such as the IP address or country of origin, certain strings or patterns in the request, the size of specific parts of the request, and the presence of malicious SQL code or scripting. You can also run [CAPTCHA puzzles and silent client session challenges](#) against requests.

Both AWS WAF and CloudFront also enable you to set [geo-restrictions](#) to block or allow requests from selected countries. This can help block or rate-limit attacks from geographic locations where you don't expect to serve users. With fine-grained geographic match rule statements in AWS WAF, you can control access down to the region level.

You can use [Scope-down statements](#) to narrow the scope of the requests that the rule evaluates to save costs and [labels on web requests](#) to allow a rule that matches the request to communicate the match results to rules that are evaluated later in the same web ACL. Choose this option to reuse the same logic across multiple rules.

We recommend using a layered funnel approach when building your web ACL rule groups, with the goal of reducing the attack vectors early. Consider adding the following rules in the order listed. After testing and evaluating the results, you can change the order or add other rules as you see fit.

Table 3 – Recommended WAF rules

WAF rule name	WAF rule type	Action	Priority
ALLOWed IPs (*)	Custom (with IP set)	ALLOW	1
BLOCKed IPs (*)	Custom (with IP set)	BLOCK	2

WAF rule name	WAF rule type	Action	Priority
GEO-fencing (*)	Custom (Country-codes)	BLOCK or ALLOW	3
AntiDDoS AMR (**)	Managed	BLOCK or COUNT	4
AWSManagedRulesAmazonIpReputationList (*) (AWSManagedIPDDoSList rule Count by Default) (*)	Managed	BLOCK during DDoS event	5
RateLimit capture-all (higher level of allowed traffic) (*)	Rate-Based rule	BLOCK, CAPTCHA, CHALLENGE	6
(multiple separate) RateLimits for specific types—such as API, URI, IP lists, J3 and J4, and so on—and lower limits for traffic type (*)	Rate-Based rule	BLOCK, CAPTCHA, CHALLENGE	7
Core rule set (*)	Managed	BLOCK, CAPTCHA, CHALLENGE	8
Bot control (common) (**)	Managed	BLOCK, CAPTCHA, CHALLENGE	9

* Basic AWS WAF pricing applies

** Available for additional fees (threat intelligence)

You can also define a complete custom response, with response code, headers, and body.

To help identify malicious requests, review your web server logs or use [AWS WAF logging](#) and request sampling. By enabling AWS WAF logging, you get detailed information about the traffic analyzed by the web ACL. AWS WAF supports log filtering, allowing you to specify which web requests are logged and which requests are discarded from the log after the inspection.

[Information recorded in the logs](#) includes the time that AWS WAF received the request from your AWS resource, detailed information about the request, and the matching action for each rule requested.

Sampled requests provide details about requests within the past three hours that matched one of your AWS WAF rules. You can use this information to identify potentially malicious traffic signatures and create a new rule to deny those requests. If you see several requests with a random query string, make sure to allow only the query string parameters that are relevant to cache for your application. This technique is helpful in mitigating a cache busting attack against your origin.

AWS WAF – Rate-based rules

AWS strongly recommends protecting against HTTP request floods by using rate-based rules in AWS WAF to automatically block or challenge requests matching a specific [aggregation key](#) (which is by IP address), when the number of requests received in a configurable sliding window exceed a threshold that you define. Client IP addresses that match the aggregation key will receive an HTTP 403 forbidden response (or custom configured block error response code or body) and remain blocked until request rates drop below the threshold.

It's recommended to layer rate-based rules to provide enhanced protection so that you have:

- A blanket rate-based rule to protect your application from large HTTP floods.
- A rule with a [custom aggregation key](#)—for example Header (Host) + URI + query string + IP—at more restrictive rates than the blanket rate-based rule. This is particularly effective against DDoS because DDoS botnets usually target the same URL repeatedly.
- A rate-based rule with JA3 or JA4 as an aggregation key to monitor request based on client TLS fingerprints.
- One or more rate-based rules to protect specific URIs—such as those that are unable to be cached—at more restrictive rates than the blanket rate-based rule.

For instance, you can choose a blanket rate-based rule (no scope-down statement) with a limit of 500 requests within a 5-minute period, and then create one or more of the following rate-based rules with lower limits than 500 (as low as 10 requests in a 10-minute period) using scope-down statements:

- Protect your **web pages** with a scope-down statement such as `if NOT uri_path contains ' . '` so that requests for resources without a file extension are further protected. This also protects your homepage (/) which is a frequently targeted URI path.

- Protect **dynamic endpoints** with a scope-down statement such as `if method exactly matches 'post'` (convert lowercase).
- Protect **heavy requests** that reach your database or invoke a one-time password (OTP) with a scope-down such as `if uri_path starts_with '/login'`, `uri_path starts_with '/signup'` or `uri_path starts_with '/forgotpassword'`.
- Use lower limits for country codes that aren't your target demographic

Rate-based rules with JA3 and JA4 headers are supported in CloudFront and ALB. Customers have asked for SSL/TLS inspection capabilities and JA3 and JA4 fingerprints to help them analyze unique TLS handshake characteristics in the form of 32 (JA3)- or 36 (JA4)-character hash fingerprint of the TLS Client Hello packet of incoming requests. By using both JA4 and JA3 fingerprinting capabilities, you can implement robust protection against automated threats while maintaining legitimate [traffic flow](#) to your applications.

Also, you can use autonomous system number (ASN)-based rate-limits to control incoming traffic from well-known ASN lists or network vendors from countries where you aren't actively operating.

Together with IP Reputation rule groups (discussed in the following section), rate-based rules in Block mode are the cornerstone of a defense-in-depth AWS WAF configuration to protect against request floods and are a requirement for [AWS Shield Advanced cost protection](#) requests to be approved.

AWS WAF – ASN matching feature

By monitoring and restricting traffic from specific ASNs, you can mitigate risks associated with malicious actors, comply with regulatory requirements, and optimize the performance and availability of your web applications. [ASN-based matching](#) simplifies WAF operations by reducing the need to use customer-maintained [IP sets](#) for similar purposes:

- You can add your trusted partners' ASNs to an ALLOW rule, for example your additional CDN vendors' ASNs in case of multi-CDN setup behind CloudFront or working on top of your ALBs
- You can improve control by introducing Captcha, Challenge, or Block of ASNs listed in the Spamhaus ASN-DROP list, which contains ASNs leased by professional SPAM or cyber-criminals
- Limit connections from countries where you don't actively operate your business by using [Geographic match rule statement](#).

AWS WAF – IP reputation rule groups

To help mitigate attacks based on IP address reputation, you can create rules that match IPs making HTTP requests to custom IPSets that you maintain or use [AWS WAF Managed IP Reputation rule groups](#). From a DDoS perspective, the two most important rules are:

- `AWSMangedIPDDoSList` rule within the `AWSMangedRulesAmazonIpReputationList` rule group
- `AnonymousIPList` rule with the `AWSMangedRulesAmazonIpReputationList` rule group

AWSMangedIPDDoSList rule within the AWSMangedRulesAmazonIpReputationList rule group: The [Amazon IP reputation list rule-group](#)

`AWSMangedRulesAmazonIpReputationList` includes rules based on various Amazon internal threat intelligence teams. These rules look for IP addresses that are bots performing reconnaissance against AWS resources or actively engaging in DDoS activities against AWS resources. The `AWSMangedIPDDoSList` rule, which is in Count mode by default, is curated by the AWS perimeter protection team and contains IPs that have recently been observed participating in DDoS attacks against our customers. Surges in traffic matching this rule are a strong indicator of a DDoS attempt and AWS recommends adding the `AWSMangedRulesAmazonIpReputationList` rule group to your web ACL and take one of the following actions with regards to the `AWSMangedIPDDoSList` rule:

- Move it into Block or Challenge mode
- Leave it in Count mode and create a subsequent rate-based rule using a custom aggregation key using the WAF label namespace `aws:waf:managed:aws:amazon-ip-list`, applied to requests matching this rule group

Choosing between these two options depends on your application's traffic pattern and DDoS resilience requirements. Based on our observations across AWS customers, web properties typically fall into two distinct categories: those that regularly receive a low, consistent volume of traffic matching the `AWSMangedIPDDoSList` rule during normal operations, and those that only see matches during active DDoS attacks. Understanding your application's baseline behavior is crucial for determining the most effective protection strategy.

If your application can handle small volumes of potentially malicious traffic without performance impact, you might opt for the rate-based approach. However, if your application is sensitive to any level of malicious traffic or you observe clear separation between legitimate and attack traffic patterns, implementing an immediate block action might be more appropriate.

Note

Even when the `AWSManagedRulesAmazonIpReputationList` rule group is in Block mode within the web ACL, the `AWSManagedIPDDoSList` rule will be in Count unless the rule group has been edited to move the rule to Block.

To determine your resources baseline, add the rule group to your web ACL and use CloudWatch AWS WAFv2 CountedRequests metrics (using the Sum statistic) for the `AWSManagedRulesAmazonIpReputationList` rule group, to identify if your web application receives traffic from this rule group during non-attack periods. You might want to use WAF logs to pivot to the specific IPs that matched rules within the rule group, to see if their behavior in terms of the requests that they made, was in keeping with what legitimate user behavior looks like. Then you can make an informed decision to Block or Challenge actions or create a rate-based rule.

If you choose to use a rate-based rule rather than block this traffic (most customers choose to block this outright) it would look like the following configuration pseudo-code:

```
rule name: rbr_ipddoslist
rule type: rate-based
limit: <a low number>
evaluation window: 300
aggregation key (custom): label namespace 'awswaf:managed:aws:amazon-ip-list:'
scope-down: if label matches 'awswaf:managed:aws:amazon-ip-list:AWSManagedIPDDoSList'
action: BLOCK
```

AnonymousIPList rule with the AWSManagedRulesAmazonIpReputationList rule group:

The [Anonymous IP list rule group](#) contains rules to block requests from services that allow the obfuscation of viewer identity. These include requests from VPNs, proxies, Tor nodes, and cloud platforms (excluding AWS). Requests from clients matching the AnonymousIPList rule are potential candidates for being used for malicious purposes like DDoS, and you should consider moving this rule to Block, but be careful of the `HostingProviderIPListinBlock`—it's a very blunt instrument and customers frequently choose to leave this rule in Count mode.

In addition, you can use third-party IP reputation lists by using the [IP Lists parser](#) component of the [Security Automations for AWS WAF](#) solution.

AWS WAF – Block access to unused URI paths and file extensions

Block access to unused URI paths using AWS WAF. The most obvious example here would be the '/' URI path for an API endpoint, which would typically return an HTTP 404 (Not Found).

Likewise consider blocking access to file extensions that you don't use, for example: `if NOT uri_path matches regex .*\. (css|js|png|jpg|svg|woff2|html|json)$ AND uri_path contains '.'`

Blocking access to such paths prevents sudden high-volume request floods on non-existent or mutating URLs from overwhelming your target group (for ALB endpoints) or origin (for CloudFront).

AWS WAF – Challenge rule action

The [Challenge rule](#) action provides an effective defense against DDoS botnets, which typically use basic scripts designed to maximize target impact while minimizing operational costs. This action requires clients to complete a JavaScript-based proof of work challenge and present a valid `aws-waf-token` cookie before allowing the request to proceed. This mechanism has proven highly effective against automated DDoS attacks, as most attack scripts can't process JavaScript or handle cookies properly.

When implementing Challenge actions, it's important to consider potential impacts on legitimate users. Some visitors might be unable to access your application if they have JavaScript disabled, use older browsers, or rely on screen readers and accessibility tools.

The token can only be acquired using HTTP GET requests with the Accept header containing text or html (unless there has been prior [client side integration](#), which can be done for customers using [AWS WAF Bot Control](#) or one of the other intelligent threat mitigations rule groups). After the token is acquired, it will be presented by the client on subsequent requests for the same domain until the TTL, in the form of cookie expiration, expires.

AWS recommends implementing Challenge actions strategically. Consider using them selectively during active attacks rather than as a continuous protection measure and combine them with rate-based rules to optimize cost efficiency.

AWS WAF – Intelligent threat mitigation

Botnets are a serious security threat and are commonly used to carry out illegal or harmful activities such as sending spam, stealing sensitive data, initiating ransomware attacks, committing ad fraud through fraudulent clicks, or launching distributed denial-of-service (DDoS) attacks. To

prevent bot attacks, use the [AWS WAF Bot Control](#) managed rule group. This rule group provides a basic, Common protection level that adds labels to self-identifying bots, verifies generally desirable bots, and detects high confidence bot signatures and a Targeted protection level that adds detection for advanced bots that don't self-identify.

Targeted protections use advanced detection techniques such as browser interrogation, fingerprinting, and behavior heuristics to identify bad bot traffic and then applies mitigation controls such as rate limiting and CAPTCHA and [Challenge rule actions](#). Targeted also provides rate limiting options to enforce human-like access patterns and apply dynamic rate limiting through the use of request tokens. For additional details, see [AWS WAF Bot Control rule group](#).

To detect and manage malicious takeover attempts on your application's login page, you can use AWS WAF Fraud Control account takeover prevention (ATP) rule group. The rule group does this by inspecting login attempts that clients send to your application's login endpoint and also inspects your application's responses to login attempts, to track success and failure rate.

Account creation fraud is an online illegal activity in which an attacker tries to create one or more fake accounts. Attackers use fake accounts for fraudulent activities such as abusing promotional and sign up bonuses, impersonating someone, and cyberattacks like phishing. The presence of fake accounts can negatively impact your business by damaging your reputation with customers and exposure to financial fraud.

You can monitor and control account creation fraud attempts by implementing the AWS WAF Fraud Control account creation fraud prevention (ACFP) feature. AWS WAF offers this feature in the AWS Managed Rules rule group `AWSMangedRu1esACFPRu1eSet` with companion [application integration](#) SDKs.

Learn more about these protections in [AWS WAF intelligent threat mitigation](#).

AWS WAF – Application layer DDoS protection

You can use the AWS WAF [AntiDDoS managed rule group](#) (Anti-DDoS AMR) to protect your web applications against DDoS events in one place when using AWS WAF. This AMR detection system is designed to distinguish between DDoS events and flash crowds, where many legitimate clients send a limited number of requests.

When added to your AWS WAF web ACL configuration, it learns your traffic patterns and establishes baselines for each protected resource. The system identifies anomalies by comparing current traffic to these baselines, assigning suspicion scores to requests for use in subsequent mitigations.

During an event, the [AntiDDoS AMR labels](#) all requests to the affected resource with an event-detected label and further identifies requests suspected to be part of the DDoS attack. The AntiDDoS AMR applies protections independently for each protected resource, basing its actions on the unique baseline established for that resource. This approach provides tailored and effective protection against DDoS attacks for your web applications.

AntiDDoS AMR starts detecting attacks after establishing a traffic baseline within minutes of AMR activation for the protected resource. Novel algorithms and techniques used in the ruleset enable time to mitigation within seconds. Furthermore, the logic inside AntiDDoS AMR uses sensitivity level and suspicion scores to minimize both false positives and false negatives to help ensure that only malicious volumetric traffic is blocked.

The AMR feature offers adjustable configuration, including [sensitivity controls](#) based on the suspicion levels detected by the system for specific events and requests from suspicious sources. This allows you to customize the protection configuration to suit the needs of your specific application types. One of the benefits of AntiDDoS AMR is its ability to detect and mitigate attacks within seconds.

To learn more about the AMR, pricing, and how to get started with it, see the [AWS WAF Distributed Denial of Service \(DDoS\) prevention rule group](#).

Third-party firewalls on Amazon EC2

While AWS WAF is the recommended solution, many customers have third-party firewalls that they want to keep using when they move to the cloud. Many major firewall providers have AMI that can be used on EC2 instances. However, these EC2-based WAFs must be carefully architected to prevent them from becoming bottlenecks during DDoS attacks.

For HTTP-based applications, AWS recommends that these firewalls are used as a secondary defense layer (fine-grained protections) behind AWS resources integrated with AWS WAF, which is configured with volumetric protection using rate-based rules and IP reputation rule groups to protect your firewall tier.

AWS Shield Advanced Features

Managed DDoS event response with Shield Response Team support

When subscribed to Shield Advanced, you can engage the AWS Shield Response Team (SRT) to help you create rules to mitigate an attack that's reducing your application's availability. For more information, see [Managed DDoS event response with Shield Response Team \(SRT\)](#).

Proactive engagement

With proactive engagement, the SRT contacts you directly when the availability or performance of your application is affected because of a possible attack. We recommend this engagement model because it provides the quickest SRT response and allows the SRT to begin troubleshooting even before they've established contact with you. Through proactive engagement, you can configure [health checks](#), associate to your resources, and provide always available operations contact information. When Shield Advanced detects signs of DDoS and your application health checks are showing signs of degradation, AWS SRT will proactively reach out to you.

Completing configuration of proactive engagement includes adding contact details in the Shield Advanced console. AWS SRT will use this information to contact you.

You can enable proactive engagement for all resources or for select key production resources where response time is critical. This is accomplished by assigning health checks only to these resources. For more information, see [Setting up proactive engagement for the SRT to contact you directly](#).

Centralized protection management

In addition, you can use AWS Firewall Manager to centrally configure and manage security rules, such as AWS Shield Advanced protections and AWS WAF rules, across your organization. Your AWS Organizations management account can designate an administrator account, which is authorized to create Firewall Manager policies. These policies allow you to define criteria, such as resource type and tags, which determine where rules are applied. This is useful when you have multiple accounts and want to standardize your protection. AWS Shield protection policies can be created using AWS Firewall Manager only for Shield Advanced users.

For more information about:

- How to manage the deployment of rules across your AWS resources with Firewall Manager, see:
 - [Getting started with Firewall Manager AWS WAF policies](#)
 - [Getting started with Firewall Manager Shield Advanced policies](#)

Attack surface reduction

Another important consideration when architecting an AWS solution is to limit the opportunities an attacker has to target your application. This concept is known as *attack surface reduction*. Resources that aren't exposed to the internet are more difficult to attack, which limits the options an attacker has to target your application's availability.

For example, if you don't expect users to directly interact with certain resources, make sure that those resources aren't accessible from the internet. Similarly, don't accept traffic from users or external applications on ports or protocols that aren't necessary for communication.

In the following section, AWS provides best practices to guide you in reducing your attack surface and limiting your application's internet exposure.

Obfuscating AWS resources (BP1, BP4, BP5)

You can limit direct exposure of your backend resources by placing edge services—such as CloudFront, API Gateway, or load balancers—between your users and your origin infrastructure. The following sections describe specific techniques for achieving this.

Security groups and network ACLs (BP5)

[Amazon Virtual Private Cloud \(Amazon VPC\)](#) allows you to provision a logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define.

Security groups and network ACLs are similar in that they allow you to control access to AWS resources within your VPC. But security groups allow you to control inbound and outbound traffic at the instance level, while network ACLs offer similar capabilities at the VPC subnet level. There is no additional charge for using security groups or network ACLs.

You can choose whether to specify security groups when you launch an instance or associate the instance with a security group later. When you first create a security group, it has no inbound rules. Therefore, no inbound traffic is allowed until you add inbound rules to the security group.

For example, when EC2 instances sit behind an Elastic Load Balancer, the instances themselves shouldn't be directly reachable from the internet and should be configured with private IP addresses only, in subnets that have no route to an internet gateway. Private addressing is what prevents direct internet reachability.

When you create network ACLs, you can specify both allow and deny rules. This is useful if you want to explicitly deny certain types of traffic to your application. For example, you can define IP addresses (as CIDR ranges), protocols, and destination ports that are denied access to the entire subnet. If your application is used only for TCP traffic, you can create a rule to deny all UDP traffic, or vice versa. This option is useful when responding to DDoS attacks because it lets you create your own rules to mitigate the attack when you know the source IPs or other signature.

If you're subscribed to [AWS Shield Advanced](#), you can register Elastic IP addresses as protected resources. DDoS attacks against Elastic IP addresses that have been registered as protected resources are detected more quickly, which can result in a faster time to mitigate. When an attack is detected, the DDoS mitigation systems reads the network ACL that corresponds to the targeted Elastic IP address and enforces it at the AWS network border, rather than at the subnet level. This significantly reduces your risk of impact from a number of infrastructure layer DDoS attacks.

For more information about configuring security groups and network ACLs to optimize for DDoS resiliency, refer to [How to Help Prepare for DDoS Attacks by Reducing Your Attack Surface](#).

For more information about using Shield Advanced with Elastic IP addresses as protected resources, refer to the steps to [Subscribe to AWS Shield Advanced](#).

Protecting your origin (BP1, BP5)

When using [Amazon CloudFront](#) with an origin inside your VPC, it's crucial to ensure that only your CloudFront distribution can forward requests to your origin. AWS provides several approaches to implement this security control, with CloudFront VPC origins being the newest and recommended solution.

CloudFront VPC origins enables you to keep your applications in private subnets without internet access while restricting access to only your CloudFront distributions. This feature allows CloudFront to deliver content directly from applications hosted in VPC private subnets, supporting [Application Load Balancers](#) (ALB), [Network Load Balancers](#) (NLB), and EC2 instances. This approach eliminates the need for public origins and creates a single, secure entry point to your web applications while maintaining the performance and scale benefits of CloudFront. There is no additional cost for using VPC origins with CloudFront.

To learn more, see [CloudFront VPC origins](#).

For origins using Amazon S3, [AWS Elemental](#), or Lambda function URLs, Origin Access Control (OAC) remains the recommended managed solution to secure these origins.

There are a number of alternative approaches for cases when you're not using VPC Origins yet, see [Restrict access to Application Load Balancers](#) for more information.

Protecting API endpoints (BP4)

When you must expose an API to the public, there's a risk that the API frontend could be targeted by a DDoS attack. To help reduce the risk, you can use [Amazon API Gateway](#) as an entryway to applications running on Amazon EC2, AWS Lambda, or other AWS services. By using Amazon API Gateway, you don't need your own servers for the API frontend and you can obfuscate other components of your application. By making it harder to detect your application's components, you can help prevent those AWS resources from being targeted by a DDoS attack.

When you use API Gateway, you can choose from two types of API endpoints. The first is the default option: edge-optimized API endpoints that are accessed through an Amazon CloudFront distribution. The distribution is created and managed by API Gateway, however, so you don't have control over it. The second option is to use a regional API endpoint that's accessed from the same AWS Region where your REST API is deployed. AWS recommends that you use the second type of endpoint and associate it with your own Amazon CloudFront distribution. This gives you control over the Amazon CloudFront distribution and the ability to use [AWS WAF](#) for application layer protection. This mode provides you with access to scaled DDoS mitigation capacity across the AWS global edge network.

When using Amazon CloudFront and AWS WAF with Amazon API Gateway, configure the following options:

- Configure the cache behavior for your distributions to forward all headers to the API Gateway regional endpoint. By doing this, CloudFront will treat the content as dynamic and skip caching the content.
- Protect your API Gateway against direct access by configuring the distribution to include the origin custom header `x-api-key`, by setting the [API key value](#) in API Gateway.
- Protect the backend from excess traffic by configuring standard or burst rate limits for each method in your REST APIs.
- Enable authentication and authorization for your APIs. Unauthenticated API endpoints are more vulnerable to application layer DDoS attacks because attackers can generate requests without needing valid credentials. Use one or more of the following authentication mechanisms:
 - IAM authorization – For service-to-service or internal API calls
 - Amazon Cognito user pools – For user-facing APIs with sign-up/sign-in flows

- Lambda authorizers – For custom authentication logic (for example, validating tokens from third-party identity providers)
- API keys – As an additional layer (not as a sole authentication mechanism)
- JWT authorizers in [API Gateway](#) – You can use JSON Web Tokens (JWTs) as a part of OpenID Connect (OIDC) and OAuth 2.0 frameworks to restrict client access to your APIs.

For more information about creating APIs with Amazon API Gateway, see [Amazon API Gateway Getting Started](#).

Operational techniques

The mitigation techniques in this paper help you architect applications that are inherently resilient against DDoS attacks. In many cases, it's also useful to know when a DDoS attack is targeting your application so you can take mitigation steps. This section discusses best practices for gaining visibility into abnormal behavior, alerting and automation, managing protection at scale, and engaging AWS for additional support.

Load testing

Regularly load test your application using the guidelines in our [Load Testing Applications](#) whitepaper with both expected and above expected traffic levels so you can see how effective your architecture is, how your Auto Scaling policies function and how your error handling functions. Test for expected traffic scale-up and down but also for flash-crowd type behavior. Retest either periodically or before any major release. For layer 3 or 4 DDoS simulation testing, such as SYN flood, follow our [DDoS Simulation Testing Policy](#).

Metrics and alarms

As a best practice you should be using infrastructure and application monitoring tools to check the availability of your application to help detect whether your application is being impacted by a DDoS event, as an option you can configure application and infrastructure Route 53 [health checks](#) for the resources to help improve the detection of DDoS events. For more information about health checks, see [AWS WAF, Firewall Manager and Shield Advanced Developer Guide](#).

For organizations operating across multiple AWS accounts, AWS recommends implementing CloudWatch [cross-account](#) observability to establish unified monitoring of DDoS-related metrics and alarms. This feature enables a central security or operations team to monitor DDoS events, analyze attack patterns, and respond to incidents across your entire organization from a single monitoring account. For more information, see [Monitor across accounts and Regions](#).

When a key operational metric deviates substantially from the expected value, an attacker might be attempting to target your application's availability. Familiarity with the normal behavior of your application means you can act more quickly when you detect an anomaly. [Amazon CloudWatch](#) can help by monitoring applications that you run on AWS. For example, you can collect and track metrics, collect and monitor log files, set alarms, and automatically respond to changes in your AWS resources.

If you follow the DDoS-resilient reference architecture when architecting your application, common infrastructure layer attacks will be blocked before reaching your application. If you're subscribed to [AWS Shield Advanced](#), you have access to a number of CloudWatch metrics that can indicate that your application is being targeted.

For example, you can configure alarms to notify you when there's a DDoS attack in progress, so you can check your application's health and decide whether to engage AWS SRT. You can configure the `DDoSDetected` metric to tell you if an attack has been detected. If you want to be alerted based on the attack volume, you can also use the `DDoSAttackBitsPerSecond`, `DDoSAttackPacketsPerSecond`, or `DDoSAttackRequestsPerSecond` metrics. You can monitor these metrics by integrating CloudWatch with your own tools or by using tools provided by third parties, such as Slack or PagerDuty.

An application layer attack can elevate many [Amazon CloudWatch](#) metrics. If you're using [AWS WAF](#), you can use CloudWatch to monitor and activate alarms on increases in requests that you've set in AWS WAF to be allowed, counted, or blocked. This allows you to receive a notification if the level of traffic exceeds what your application can handle. You can also use [Amazon CloudFront](#), [Amazon Route 53](#), Application Load Balancer, [Network Load Balancer](#), Amazon EC2, and Auto Scaling metrics that are tracked in CloudWatch to detect changes that can indicate a DDoS attack.

The following table lists descriptions of CloudWatch metrics that are commonly used to detect and react to DDoS attacks.

Table 4 – Recommended Amazon CloudWatch metrics

AWS service	Metric	Description
AWS Shield Advanced	<code>DDoSDetected</code>	Indicates a DDoS event for a specific Amazon Resource Name (ARN).
AWS Shield Advanced	<code>DDoSAttackBitsPerSecond</code>	The number of bytes observed during a DDoS event for a specific ARN. This metric is only available for layer 3 or 4 DDoS events.
AWS Shield Advanced	<code>DDoSAttackPacketsPerSecond</code>	The number of packets observed during a DDoS

AWS service	Metric	Description
		event for a specific ARN. This metric is only available for layer 3 or 4 DDoS events.
AWS Shield Advanced	DDoSAttackRequestsPerSecond	The number of requests observed during a DDoS event for a specific ARN. This metric is only available for layer 7 DDoS events and is only reported for the most significant layer 7 events.
AWS WAF	AllowedRequests	The number of allowed web requests.
AWS WAF	BlockedRequests	The number of blocked web requests.
AWS WAF	CountedRequests	The number of counted web requests.
AWS WAF	PassedRequests	The number of passed requests. This is only used for requests that go through a rule group evaluation without matching any of the rule group rules.
Amazon CloudFront	Requests	The number of HTTP/S requests.
Amazon CloudFront	TotalErrorRate	The percentage of all requests for which the HTTP status code is 4xx or 5xx.
Amazon Route 53	HealthCheckStatus	The status of the health check endpoint.

AWS service	Metric	Description
Application Load Balancer	ActiveConnectionCount	The total number of concurrent TCP connections that are active from clients to the load balancer, and from the load balancer to targets.
Application Load Balancer	ConsumedLCUs	The number of load balancer capacity units (LCU) used by your load balancer.
Application Load Balancer	HTTPCode_ELB_4XX_Count HTTPCode_ELB_5XX_Count	The number of HTTP 4xx or 5xx client error codes generated by the load balancer.
Application Load Balancer	NewConnectionCount	The total number of new TCP connections established from clients to the load balancer, and from the load balancer to targets.
Application Load Balancer	ProcessedBytes	The total number of bytes processed by the load balancer.
Application Load Balancer	RejectedConnectionCount	The number of connections that were rejected because the load balancer had reached its maximum number of connections.
Application Load Balancer	RequestCount	The number of requests that were processed.

AWS service	Metric	Description
Application Load Balancer	TargetConnectionErrorCount	The number of connections that were not successfully established between the load balancer and the target.
Application Load Balancer	TargetResponseTime	The time elapsed, in seconds, after the request left the load balancer until a response from the target is received.
Application Load Balancer	UnHealthyHostCount	The number of targets that are considered unhealthy.
Network Load Balancer	ActiveFlowCount	The total number of concurrent TCP flows (or connections) from clients to targets.
Network Load Balancer	ConsumedLCUs	The number of load balancer capacity units (LCU) used by your load balancer.
Network Load Balancer	NewFlowCount	The total number of new TCP flows (or connections) established from clients to targets in the time period.
Network Load Balancer	ProcessedBytes	The total number of bytes processed by the load balancer, including TCP/IP headers.
Global Accelerator	NewFlowCount	The total number of new TCP and UDP flows (or connections) established from clients to endpoints in the time period.

AWS service	Metric	Description
Global Accelerator	ProcessedBytesIn	The total number of incoming bytes processed by the accelerator, including TCP/IP headers.
Auto Scaling	GroupMaxSize	The maximum size of the Auto Scaling group.
Amazon EC2	CPUUtilization	The percentage of allocated EC2 compute units that are currently in use.
Amazon EC2	NetworkIn	The number of bytes received by the instance on all network interfaces.

For more information about using Amazon CloudWatch to detect DDoS attacks on your application, refer to [Getting Started with Amazon CloudWatch](#).

AWS includes several additional metrics and alarms to notify you about an attack and to help you monitor your application's resources. The [AWS Shield](#) console or API provide a per-account event summary and details about attacks that have been detected.

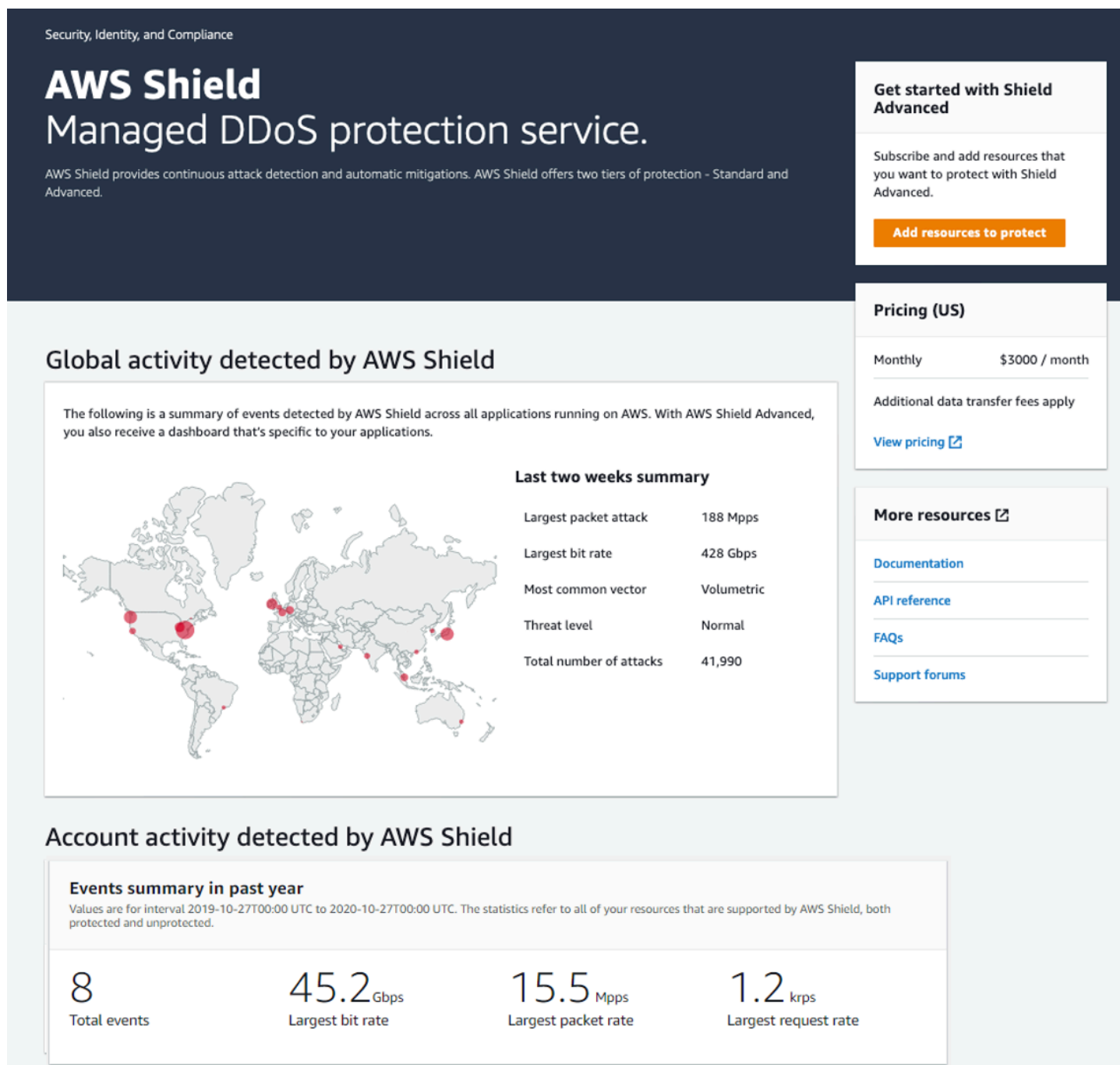


Figure 11 – Global activity detected by AWS Shield

In addition, the global threat environment dashboard provides summary information about all DDoS attacks that have been detected by AWS. This information can be useful to better understand DDoS threats across a larger population of applications in addition to attack trends, and comparing with attacks that you might have observed.

If you're subscribed to AWS Shield Advanced, the service dashboard displays additional detection and mitigation metrics and network traffic details for events detected on protected resources. AWS Shield evaluates traffic to your protected resource along multiple dimensions. When an anomaly is detected, AWS Shield creates an event and reports the traffic dimension where the anomaly was observed. With a placed mitigation this protects your resource from receiving excess traffic and traffic that matches a known DDoS event signature.

Detection metrics are based on sampled network flows or AWS WAF logs when a web ACL is associated with the protected resource. Mitigation metrics are based on traffic that's observed by [AWS Shield](#)'s DDoS mitigation systems. Mitigation metrics are a more precise measurement of the traffic into your resource.

The *network top contributors* metric provides insight into where traffic is coming from during a detected event. You can view the highest volume contributors and sort by aspects such as protocol, source port, and TCP flags. The top contributors metric includes metrics for all traffic observed on the resource along various dimensions. It provides additional metric dimensions you can use to understand network traffic that's sent to your resource during an event. Keep in mind that for non-reflection layer 3 or 4 attacks, the source IP addresses could have been spoofed and can't be relied on.

The service dashboard also includes details about the actions automatically taken to mitigate DDoS attacks. This information makes it easier to investigate anomalies, explore dimensions of the traffic, and better understand the actions taken by Shield Advanced to protect your availability.

AWS WAF traffic overview dashboard

The traffic overview dashboard in AWS WAF displays an overview of security-focused metrics so that you can identify and act on security risks in a few clicks, such as adding [rate-based rules](#) during DDoS events. The dashboards include near real-time summaries of the CloudWatch metrics that AWS WAF collects when it evaluates your applications' web traffic. These dashboards are available by default and require no additional setup. They show metrics for each web ACL that you monitor with AWS WAF, including total requests, blocked requests, allowed requests, bot compared to non-bot requests, bot categories, CAPTCHA solve rate, top 10 matched rules, and more.

You can access default metrics such as the total number of requests, blocked requests, and common attacks blocked, or you can customize your dashboard with the metrics and visualizations that are most important to you.

These dashboards provide enhanced visibility and help you answer questions such as:

- What percent of the traffic that AWS WAF inspected is getting blocked?
- What are the top originating countries for the traffic that's being blocked?
- What are common attacks that AWS WAF detects and protects me from?
- How do my traffic patterns from this week compare with last week?

The dashboard has built-in integration with CloudWatch. Using this integration, you can navigate back and forth between the dashboard and CloudWatch; for example, you can get a more granular metric overview by viewing the dashboard in CloudWatch. You can also add existing CloudWatch widgets and metrics to the traffic overview dashboard, bringing your tried-and-tested visibility structure into the dashboard.

Logging

Enable useful logging on all services according to our [Logging and monitoring guide for application owners](#) to maximize visibility and assist with troubleshooting. This includes, but isn't limited to:

- [AWS CloudTrail](#)
- [AWS WAF Logs](#)
- [CloudFront access logs](#)
- [VPC Flow Logs](#) (see [Log and View Network Traffic Flows](#)) – include tcp-flags field in the included fields to maximize visibility
- ELB access logs ([ALB](#), [NLB](#))
- [API Gateway](#) logs
- Web server HTTP access logs
- Operating system security logging
- [Application logging](#)

Log correlation for DDoS resilient architectures

For DDoS resilient architectures, it's crucial to be able to correlate the logs between the services in the scope. For a typical web application setup, those services can be CloudFront, AWS WAF, and ALB.

When CloudFront forwards an HTTP request to its origin, it automatically injects the `X-Amz-Cf-Id` header, which contains an opaque string uniquely identifying the request. This value is logged in the CloudFront access logs under the `x-edge-request-id` field.

Similarly, ALB automatically injects the `X-Amzn-Trace-Id` header when forwarding a request to the target group. This header's value is logged in the ALB access logs under the `trace_id` field.

When a CloudFront distribution has a WAF web ACL attached, the WAF includes a `requestId` field in the WAF logs that contains the value of `X-Amz-Cf-Id` from CloudFront.

To see the relationship of the three services, see the following table:

CloudFront logs	WAF (CloudFront)	ALB logs	Backend server log
<code>X-Amz-Cf-Id = 'Ed0AiHF_CGYF-DA='</code>	<code>requestId = 'Ed0AiHF_CGYF-DA='</code>	<code>X-Amzn-Trace-Id = 'Root=1-67891233-abcdef012345678912345678'</code>	<code>X-Amz-Cf-Id = Ed0AiHF_CGYF-DA= X-Amzn-Trace-Id = 'Root=1-67891233-abcdef012345678912345678'</code>

Similarly, when you attach a WAF web ACL to ALB, you will have `X-Amzn-Trace-Id` in the WAF logs that you can correlate with an ALB log value. For WAF logs examples, see [Log examples for web ACL traffic](#).

WAF (ALB)	ALB logs	Backend server Log
<code>X-Amzn-Trace-Id = 'Root=1-67891233-abcdef012345678912345678'</code>	<code>X-Amzn-Trace-Id = 'Root=1-67891233-abcdef012345678912345678'</code>	<code>X-Amzn-Trace-Id = 'Root=1-67891233-abcdef012345678912345678'</code>

For useful examples of WAF queries using Athena, see [How to use Amazon Athena queries to analyze AWS WAF logs and provide the visibility needed for threat detection](#).

Visibility and protection management across multiple accounts

In scenarios when you operate across multiple AWS accounts and have multiple components to protect, use techniques that enable you to operate at scale and reduce operational overhead to increase your mitigation capabilities.

[AWS Firewall Manager](#) simplifies your administration and maintenance tasks across multiple accounts and resources for a variety of protections, including AWS WAF, AWS Shield Advanced, Amazon VPC security groups and network ACLs, AWS Network Firewall, and Amazon Route 53 Resolver DNS Firewall. You can use Firewall Manager to centrally set up and monitor your resources' compliance against your company policy and to semi-automatically apply security mitigations to protected resources. For example, you can set up recommended WAF web ACL rule groups for DDoS compliance that should be used across all or specifically tagged CloudFront distributions or ALBs, and Firewall Manager will monitor and be able to auto-mitigate the ones that are non-compliant.

When managing AWS Shield Advanced protected resources in multiple accounts, you can set up centralized monitoring by using AWS Firewall Manager and [AWS Security Hub CSPM](#). Security Hub CSPM automatically integrates with Firewall Manager, allowing Shield Advanced customers to view security findings in a single dashboard, alongside other high priority security alerts and compliance statuses.

As of writing, the Shield Advanced event doesn't appear in the Firewall Manager central account in real-time—there can be a delay of up to 15 minutes, so it's not recommended to rely on this feature for real-time notifications.

For example, when Firewall Manager evaluates accounts within the scope of a Shield Advanced policy and finds a resource that isn't yet protected by Shield Advanced, it generates a non-compliance. If the policy is configured for automatic remediation, Firewall Manager brings the resource into compliance by associating Shield Advanced protection with it. After the resource is protected, Shield Advanced begins establishing a traffic baseline for it, and any subsequently detected anomalous traffic is surfaced as a Shield Advanced finding in Security Hub CSPM, alongside CloudWatch metrics and events.

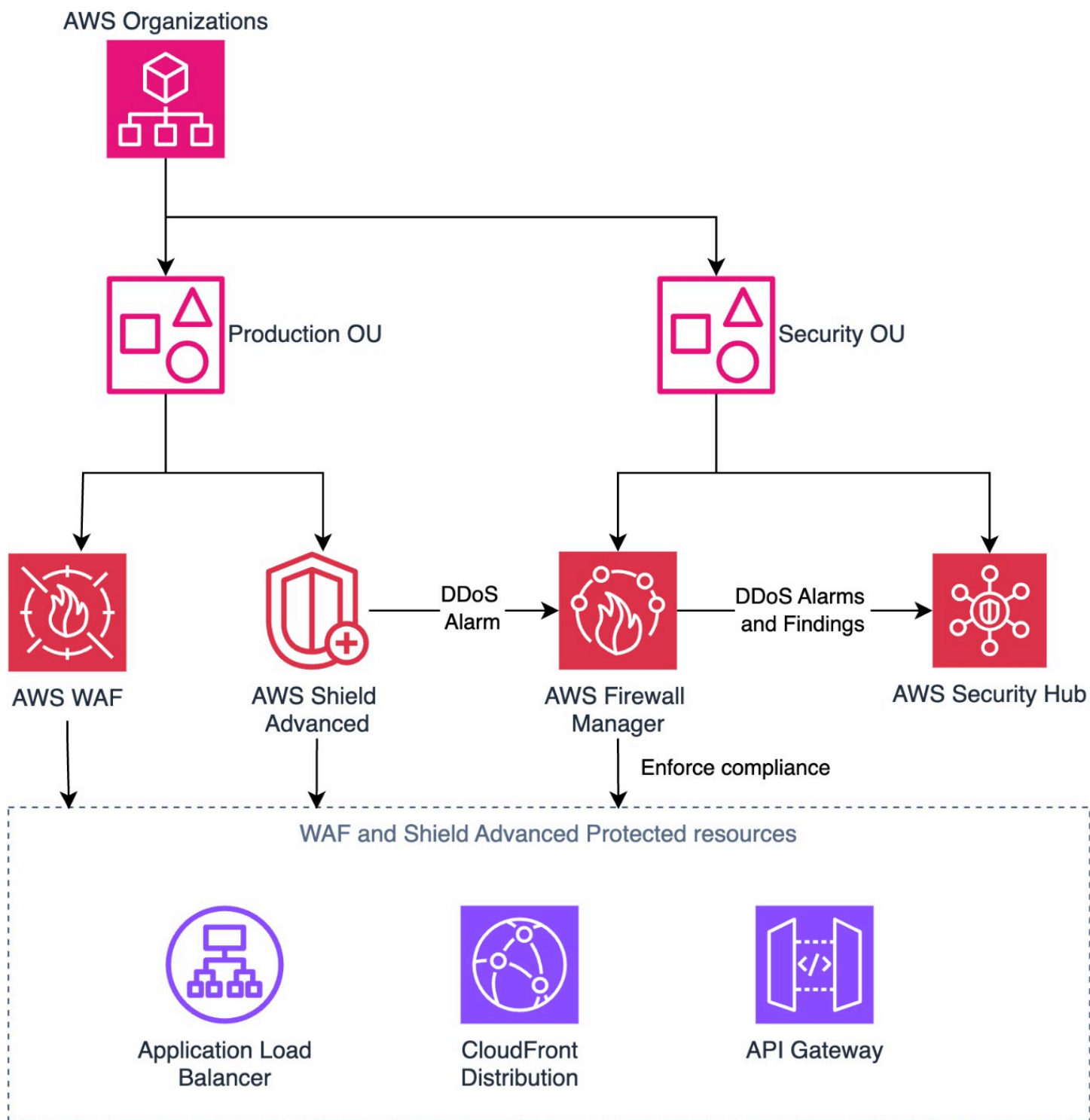


Figure 12 – Architecture diagram showing monitoring AWS Shield-protected resources with Firewall Manager and Security Hub CSPM

For more information about central monitoring of Shield protected resources, refer to [Set up centralized monitoring for DDoS events and auto-remediate noncompliant resources](#).

Incident response strategy and runbooks

Developing a DDoS attack incident response strategy and building a security incident response process around it is crucial for all organizations. A recommended approach is to model your response playbook based on NIST's suggested steps such as gathering evidence, mitigating, recovering, and conducting post-incident analysis. For example, a response playbook for web application DoS or DDoS attacks is provided as an [example](#). Additional resources are available in the [AWS Security Incident Response Guide](#).

Support

If you experience an attack, you can also benefit from support from AWS in assessing the threat and reviewing the architecture of your application, or you might want to request other assistance. It is important to create a response plan for DDoS attacks before an actual event. The best practices outlined in this paper are intended to be proactive measures that you implement before you launch an application, but DDoS attacks against your application might still occur. Review the options in this section to determine the support resources that are best suited for your scenario.

Your account team can evaluate your use case and application and assist with specific questions or challenges that you have.

If you're running production workloads on AWS, consider subscribing to Business Support+, which provides 24/7 access to Cloud Support Engineers and a response time of less than 30 minutes for business-critical system outages. For mission-critical workloads, consider Enterprise Support for a designated Technical Account Manager and less than 15-minute response times, or Unified Operations for designated Domain Specialist Engineers, Incident Management Engineers, and always available workload monitoring with less than 5-minute response times for critical issues.

For more information, refer to [Compare Support Plans](#).

If you're subscribed to AWS Shield Advanced and are also subscribed to either Business Support +, Enterprise Support, or Unified Operations, you can also escalate to AWS Shield Response Team (SRT) by creating an Support case under AWS Shield using the [Support console](#) (sign-in required), or the [support API](#) when a DDoS-related event affects your application's availability.

Conclusion

DDoS resilience isn't a one-time project but an ongoing practice of applying the right architecture, services, and operational readiness before an attack occurs. Throughout this whitepaper, AWS outlined a DDoS-resilient reference architecture, explained how infrastructure layer and application layer attacks differ, and mapped the AWS services and best practices that mitigate each. Adopting these recommendations helps you protect application availability, preserve customer trust, and avoid the unnecessary scaling costs that attacks can drive.

The most important takeaway is also the most actionable: AWS Shield Response Team data shows that most customers who suffer business impact from DDoS attacks had not yet implemented the guidance in this paper. Review your applications against this guidance today, close the gaps where you find them, and treat DDoS resilience as a continuous part of your security posture, so that when an attack comes, your architecture is already prepared to absorb and mitigate it.

Contributors

The following individuals contributed to this guide.

Authors

- Rodrigo Ferroni, AWS Principal Security Specialist
- Dmitriy Novikov, AWS Senior Solutions Architect
- Joanna Knox, AWS Senior Cloud Support Engineer

Reviewers

- Chen Andrew, AWS Shield PMT
- Maitreya Ranganath, AWS Principal Security SA

Previous contributors

- Achraf Souk, AWS Principal Edge Specialist SA
- Anuj Butail, AWS Solutions Architect
- Harith Gaddamanugu, AWS Edge Specialist SA

Further reading

For additional information, see:

- [NET318 – re:Invent 2025: From threat to threat intel: 360 degrees of DDoS](#) (YouTube video)
- [CDN306 – re:Invent 2024: You are under a DDOS attack! Are you ready to respond?](#) (YouTube video)
- [SEC344 – re:Invent 2024: Chalk Talk: Lessons learned from DDoS: Insights from AWS SRT escalations.](#) (PDF presentation)
- [NIS301 – re:Inforce 2023: How AWS threat intelligence becomes managed firewall rules](#) (YouTube video)
- [NET314 – re:Invent 2022: Building DDoS-resilient applications using AWS Shield](#) (YouTube video)
- [SEC321 – re:Invent 2020: Get ahead of the curve with DDoS Response Team escalations](#) (YouTube video)
- [William Hill: High-performance DDoS Protection with AWS: 2020](#) (YouTube video)
- [SEC407 – re:Invent 2019: A defense-in-depth approach to building web applications](#) (YouTube video)
- [Best Practices for DDoS Mitigation on AWS](#) – 2018 (YouTube video)
- [SID324 – re:Invent 2017: Automating DDoS Response in the Cloud](#) (YouTube video)
- [CTD304 – re:Invent 2017: Dow Jones & Wall Street Journal's Journey to Manage Traffic Spikes While](#) (YouTube video)
- [Mitigating DDoS & Application Layer Threats](#) (YouTube video)
- [CTD310 – re:Invent 2017: Living on the Edge, It's Safer Than You Think! Building Strong with Amazon](#) (YouTube video)
- [CloudFront, AWS Shield, and AWS WAF](#) (YouTube video)

Document revisions

To be notified about updates to this whitepaper, subscribe to the RSS feed.

Change	Description	Date
Whitepaper update	Major revision. Added comprehensive AntiDDoS AWS Managed Rules (AntiDDoS AMR) coverage. Enhanced JA3 and JA4 fingerprinting capabilities and implementation guidance. Expanded CDN caching strategies and request collapsing best practices. New reference architectures for web, TCP and UDP, and on-premises scenarios. Updated WAF configuration examples with custom aggregation keys. Added detailed log correlation guidance for DDoS resilient architectures. Refreshed technical content to reflect latest AWS service capabilities and integration patterns.	August 2026
Whitepaper update	Added OAC for CloudFront and DNS wildcard cost protection. Expanded discussion of operational techniques, caching, rate-based rules and managed rule groups. Added on-premises into architecture diagram, removed duplication, and clarified text to remove ambiguity.	August 9, 2023
Whitepaper update	Revised for clarity; Updated to include latest recommendations and features: Security group connection tracking and Shield Advanced automatic application layer DDoS mitigation.	April 13, 2022
Whitepaper update	Updated to include latest recommendations and features. AWS Global Accelerator is added as part of comprehensive protection at the edge. AWS Firewall Manager for centralized monitoring for DDoS events and auto-remediate non-compliant resources.	September 21, 2021

Change	Description	Date
Whitepaper update	Updated to clarify cache busting in Detect and Filter Malicious Web Requests (BP1, BP2) section, and ELB and ALB usage in Scale to Absorb (BP6) section. Updated diagrams and Table 2, marked "Choice of Region." as BP8. Updated BP7 section with more details.	December 18, 2019
Whitepaper update	Updated to include AWS WAF logging as a best practice.	December 1, 2018
Whitepaper update	Updated to include AWS Shield, AWS WAF features, AWS Firewall Manager, and related best practices.	June 1, 2018
Whitepaper update	Added prescriptive architecture guidance and updated to include AWS WAF.	June 1, 2016
Initial publication	Whitepaper published.	June 1, 2015

Notices

Customers are responsible for making their own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) represents current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. AWS products or services are provided "as is" without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

© 2026 Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS Glossary

For the latest AWS terminology, see the [AWS glossary](#) in the *AWS Glossary Reference*.